

Время: 00:00–06:00

ЭКРАН: доска, зона «Что было в первой части» → карта текущего выпуска → текстовая карточка DEVOPS MAY CRY.

ГОВОРЮ

«В первом видео кластер уже был готов. На нём мы разобрали архитектуру Kubernetes, основные объекты и посмотрели, что происходит при `kubectl apply`. Но я не показывал, откуда берутся сами машины, кто ставит на них Kubernetes и чем self-managed кластер отличается от managed. Путь запроса от браузера до Pod'a тоже остался на продолжение — к нему мы придём в четвёртой части этого выпуска.

Сегодня закрываем именно пробел вокруг платформы. Начнём с маленьких примеров Terraform и Ansible, затем посмотрим исходники Kubespray, создадим инфраструктуру кодом, соберём self-managed кластер, рядом поднимем managed и сравним их не по рекламной таблице, а по тому, что реально видно в терминале.

Для лабораторий у нас одно сквозное приложение — DEVOPS MAY CRY. Это вымышленная ночная диспетчерская: принимает заявки о странных инцидентах, хранит их в Postgres и умеет создавать нагрузку для следующих частей. Название — отсылка к Devil May Cry; история диспетчерской и техническая модель придуманы для курса. Технически это обычный Go-сервис с Dockerfile, Compose, probes, ресурсами и отдельными Kubernetes-манифестами. Поэтому taints, OOM, HPA, DNS и внешний вход дальше проверяем на одном приложении, а не на сорока случайных nginx.

Сейчас ничего не запускаю вслепую. Сначала фиксирую корень клона и локальные параметры записи. Секреты и реальные inventory в Git не попадают».

🖥️ ПОКАЗАТЬ

- Корень чистого клона, а не папку подготовки видео.
- `apps/devops-may-cry`: Go-код, Dockerfile, Compose, migrations.
- `kubernetes/devops-may-cry`: base, scheduling, scaling, services.
- `recording/recording.env.example`: только названия переменных; реальный `.recording.env` не раскрывать.

📋 V2-C1-S01-C01 · Зафиксировать корень клона и параметры записи

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"
test -f recording/.recording.env || cp recording/recording.env.example
recording/.recording.env
set -a
source recording/.recording.env
set +a
printf 'repo=%s\nself=%s\nmanaged=%s\n' "$REPO_ROOT" "$SELF_CONTEXT"
"$MANAGED_CONTEXT"
```

✅ После команды

Ожидаю: абсолютный путь текущего клона и контексты `kubespray` / `yc-managed`. Если env-файл только что создан, остановись до REC и заполни его.

🧠 ЧТО ЭТО ЗНАЧИТ

Репозиторий переносим между машинами без абсолютных путей. Публичный код хранит только примеры и структуру; локальные адреса, ключи, inventory и state остаются вне Git.

⚠ ЕСЛИ НЕ СОВПАЛО

Если `git rev-parse` не сработал — открыт не клон репозитория. Если env-файл только создан — стоп, заполнить вне кадра и заново начать сцену.

ПЕРЕХОД

«Сначала два маленьких примера. После них в большом Kubespray-проекте будут знакомы inventory, playbook, role и идемпотентность».

с1.1 Terraform: код описывает желаемое состояние

Время: 06:00–17:00

ЭКРАН: VS Code, `01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform`.

ГОВОРЮ

«Terraform не исполняет список шагов сверху вниз как shell-скрипт. Он читает конфигурацию, строит граф зависимостей, сравнивает желаемое состояние с тем, что знает из state, и предлагает план.

Открываю `main.tf`. Здесь специально нет облака: провайдер `local` создаёт два обычных файла — журнал и сводку смены. Так можно увидеть весь цикл без ожидания VM и без счёта за ресурсы.

`terraform init` загружает провайдер и создаёт lock-файл. В боевом репозитории lock-файл коммитят: он фиксирует выбранную версию провайдера. `plan` пока ничего не меняет. Он показывает две операции create».

ПОКАЗАТЬ

- `terraform { required_providers }`.
- `variable "dispatcher"`.
- Два `local_file` и зависимость сводки от журнала.
- `output`, не раскрывая никакие секреты.

V2-C1-S02-C01 · Terraform init и plan на двух обычных файлах

```
cd "$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform"
rm -f zhurnal.txt svodka.txt terraform.tfstate terraform.tfstate.backup
terraform init
terraform plan
```

✓ После команды

Ожидая: `Plan: 2 to add, 0 to change, 0 to destroy`. В кадре показываем `main.tf` до запуска.

🗣 ГОВОРЮ

«Теперь применяю план. На интерактивном подтверждении Terraform принимает полное `yes`. После `apply` сверяю не только зелёную строку, но и реальные файлы. Повторный `plan` должен быть пустым — это базовая проверка, что код и состояние совпадают».

📄 V2-C1-S02-C02 · Terraform apply, state и идемпотентность

```
terraform apply
cat zhurnal.txt
cat svodka.txt
terraform plan
terraform output
```

✓ После команды

Ожидая: после `yes` создаются два файла; повторный `plan` сообщает `No changes`.

🗣 ГОВОРЮ

«Теперь имитирую ручную правку мимо кода. Меняю журнал, запускаю `plan` — Terraform видит расхождение и предлагает вернуть ресурс к описанному состоянию. Это `drift`. В реальном проекте такой `plan` часто первым показывает, что кто-то подкрутил облако руками.

Важный нюанс: Terraform не всеведущий. Он сравнивает код, state и данные провайдера. Потерянный или общий локальный state — риск. В команде state обычно лежит в удалённом backend с блокировкой и контролем доступа».

☰ V2-C1-S02-C03 · Дрифт, восстановление из кода и cleanup

```
echo "тут никого не было" > zhurnal.txt
terraform plan
terraform apply
head -2 zhurnal.txt
terraform destroy
```

✅ После команды

Ожидая: Terraform видит ручную правку как расхождение, восстанавливает объявленное содержимое, затем удаляет свои два ресурса.

🧠 ЧТО ЭТО ЗНАЧИТ

`resource` описывает объект, `variable` — вход, `output` — интерфейс наружу, `state` — связь между кодом и реальными объектами. `plan` показывает намерение, `apply` меняет среду.

⚠️ НЕ ГОВОРИТЬ

Не называть state «единственным источником истины»: желаемое состояние — код, соответствие реальным объектам Terraform отслеживает через state и refresh провайдера.

ПЕРЕХОД

«Terraform создаёт инфраструктуру. Но внутри новой Linux-машины Kubernetes сам не появится. Для настройки машин нужен другой слой».

c1.2 Ansible: привести машины к нужному состоянию

Время: 17:00–29:00

ЭКРАН: 01_ЧАСТЬ_1_КЛАСТЕР/02_1.2_hello_ansible.

ГОВОРЮ

«Ansible берёт inventory, подключается к хостам и выполняет модули. В обычном Linux-сценарии агент на сервер ставить не надо: достаточно SSH и Python. Здесь первый пример локальный, поэтому в inventory явно написано

`ansible_connection=local`. Это учебная замена SSH, а не удалённый сервер.

Открываю `playbook.yml`. `hosts` выбирает группу, `vars_files` подключает тарифы, `tasks` описывают состояние файлов, а handler запускается только когда задача сообщила `changed`.

Сначала очищаю результат прошлого дубля, затем проверяю inventory и запускаю playbook».

ПОКАЗАТЬ

- `inventory.ini` и локальный тип соединения.
- `vars/prices.yml`.
- `playbook.yml`: template/copy/file, notify и handlers.

V2-C1-S03-C01 · Ansible inventory, ping и первый play

```
cd "$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/02_1.2_hello_ansible"
rm -rf /tmp/night_office
cat inventory.ini
ansible -i inventory.ini all -m ansible.builtin.ping
ansible-playbook -i inventory.ini playbook.yml
cat /tmp/night_office/prays.txt
```

✅ После команды

Ожидая: localhost отвечает `pong`, первый `PLAY RECAP` даёт `failed=0` и `changed=4`.

🗣 ГОВОРЮ

«Первый запуск изменил четыре вещи. Запускаю тот же playbook второй раз. Если состояние уже нужное, модули не делают лишнюю работу».

📄 V2-C1-S03-C02 · Второй play ничего не меняет

```
ansible-playbook -i inventory.ini playbook.yml
```

✅ После команды

Ожидая: `changed=0` и handler не запускается. Это и есть проверяемая идемпотентность.

🗣 ГОВОРЮ

«Теперь меняю одну входную переменную. Ansible обновляет только зависящий от неё файл и запускает handler. Это полезнее, чем всегда выполнять `systemctl restart`: сервис перезапускается только после реального изменения конфигурации».

📄 V2-C1-S03-C03 · Изменение переменной вызывает только нужные изменения

```
sed -i '' 's/: 4000/: 6000/' vars/prices.yml
ansible-playbook -i inventory.ini playbook.yml
cat /tmp/night_office/rassylka.txt
sed -i '' 's/: 6000/: 4000/' vars/prices.yml
rm -rf /tmp/night_office
```


✅ После команды

Ожидая: `changed=2`, handler срабатывает, итоговая сумма меняется; в конце исходное значение возвращено.

🧠 ЧТО ЭТО ЗНАЧИТ

Inventory отвечает «где», playbook — «какой сценарий», role — «какой переиспользуемый компонент», module — «какое состояние получить». Идемпотентность — проверяемый второй запуск с `changed=0`, а не обещание в README.

🏭 В ПРОДЕ

Inventory может быть динамическим, секреты идут из Vault/secret manager, а запуск — из CI или runner с ограниченным доступом. Если локально playbook работает, а в CI получает 403, сначала проверяют identity, route, allowlist и секреты; пустые логи приложения в такой ситуации ничего не говорят, потому что приложение могло вообще не стартовать.

ПЕРЕХОД

«Теперь открываем Kubespray. Это не отдельный бинарник-установщик, а большой Ansible-проект».

Время: 29:00–39:00

ЭКРАН: `kubespray` и pinned upstream `kubespray/vendor/kubespray`.

ГОВОРЮ

«Мы закрепили Kubespray на теге `v2.31.0`. В свой репозиторий не копируем весь чужой проект и не зависим от случайной ветки master: сохраняем версию, свой inventory, group_vars и загрузочный backstage-механизм.

Открываю верхний `cluster.yml`. Он импортирует playbooks. Дальше обычные роли: container engine, etcd, control plane, worker, network plugin. Наши решения лежат не в переписанном Kubespray, а в inventory и group_vars».

V2-C1-S04-C01 · Kubespray — обычный большой Ansible-проект

```
cd "$REPO_ROOT"
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
cat "$KUBESPRAY_ROOT/VERSION"
sed -n '1,180p' "$UPSTREAM/cluster.yml"
find "$UPSTREAM/roles" -mindepth 1 -maxdepth 1 -type d -print | sort | sed -n
'1,24p'
sed -n '1,180p' "$KUBESPRAY_ROOT/inventory/video2/inventory.example.ini"
sed -n '1,180p' "$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/k8s-
cluster.yml"
```

После команды

Ожидая: pinned tag `v2.31.0`, верхнеуровневый `cluster.yml`, обычные роли, inventory и group_vars. Здесь ничего не устанавливаем.

ПОКАЗАТЬ

- `kubespray/VERSION`.
- `vendor/kubespray/cluster.yml`.
- Несколько ролей, не листать весь каталог.
- `inventory.example.ini`: группы нод.
- `group_vars/k8s_cluster/k8s-cluster.yml`: версия, runtime, CNI, CIDR, strictARP.

ЧТО ЭТО ЗНАЧИТ

Kubespray использует kubeadm под капотом, но добавляет проверенную структуру ролей и настройки для нескольких дистрибутивов. Это всё ещё Ansible: inventory → variables → roles → playbooks.

ЛОВУШКА

Не начинать установку здесь. Сначала нужны машины и проверенный inventory.

ПЕРЕХОД

«Посмотрели, чем машины будут настроены. Теперь разберёмся, откуда сами машины берутся».

Время: 39:00–55:00 плюс создание стенда

ЭКРАН: обычный урок BoostMentor → пять VM → форма создания VM в Yandex Cloud.

ГОВОРЮ

«Сначала беру рабочий стенд курса. Открываю обычный урок

<https://boostmentor.ru/dashboard/courses/devops-school/lesson/g8-project-cluster-lab1>, перехожу в практику и запускаю одно окружение. Оно создаёт пять чистых VM: node1–node3 для Kubespray и lb1–lb2 для внешнего HA. Перед стартом смотрю реальный таймер в UI и выбираю максимальный доступный срок. Называю в кадре только то, что фактически показано.

У задания внутри урока есть свой самостоятельный сценарий. В этом видео от платформы нужен только инфраструктурный контракт: пять чистых VM, SSH-адреса и ключ. Роли нод, containerd, версии, сети и Calico задаёт наш pinned inventory с group_vars — его я сейчас и показываю.

После нажатия не заполняю эфир ожиданием. Говорю “АНТРАКТ”, останавливаю REC, жду готовность пяти VM, скачиваю ключ и вне захвата заново ввожу пять SSH-адресов в `recording/.recording.env`. Возвращаюсь только когда SSH доступен. Если до конца таймера меньше восьми часов, запись Ч1 делю в два захода по чекпойнту перед C1.4-бис; не надеюсь, что среда сама проживёт дольше заявленного срока.

Теперь открываю облачную консоль. Одну VM можно создать руками: выбрать образ, CPU, память, диск, сеть, security group, SSH-ключ. Я пройду форму и не нажму кнопку создания. Для пяти машин ручной путь уже неудобен: легко получить разные диски, разные правила или забыть, почему одна нода отличается.

Terraform фиксирует те же решения в коде. UI полезен, чтобы увидеть реальные ресурсы и проверить результат. Но постоянные изменения должны возвращаться в код, иначе следующий plan покажет drift».

«Перед Terraform обновляю короткоживущий IAM-токен из активного профиля `yc`. Token, cloud ID и folder ID передаются провайдеру только через окружение текущего терминала: в код, state и Git они не попадают. Сначала один безопасный read-запрос к Compute Cloud — если он не проходит, apply не начинаю».

V2-C1-S05-C01 · До Terraform проверить облачный UI без второго ручного стенда

```
export YC_TOKEN="$(yc iam create-token)"
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN" && test -n "$YC_CLOUD_ID" && test -n "$YC_FOLDER_ID"
yc compute instance list --folder-id "$YC_FOLDER_ID" --format json \
  | jq -r '.[[] | [.name, .status] | @tsv'
printf 'Yandex CLI and Terraform environment: OK\n'
```

После команды

Ожидаю: свежий IAM-токен остаётся только в окружении текущего терминала, CLI отвечает, затем виден либо пустой список, либо существующие учебные VM. В браузере руками открываем Compute Cloud и только показываем поля создания VM — второй комплект ресурсов кликами не создаём.

ПОКАЗАТЬ В БРАУЗЕРЕ

- BoostMentor → курс DevOps → урок `g8-project-cluster-lab1` → «Открыть лабораторию».
- Одна кнопка запуска; после ожидания карточки node1–node3 и lb1–lb2.
- Yandex Compute Cloud → Create VM.
- Ubuntu 24.04, CPU/RAM/disk.
- Network interface, private/public IP, security group, SSH public key.

- Уйти без создания второй ручной VM.



В ПРОДЕ

Ручное создание иногда используют для диагностики или временного ресурса, но затем либо удаляют его, либо backport'ят изменение в Terraform. Иначе репозиторий перестаёт описывать среду.

ПЕРЕХОД

«Открываю self-managed Terraform и иду по структуре файлов, а не по одной портянке».

Время: 55:00–78:00 плюс ожидание apply/destroy

ЭКРАН: `infra/self-managed/terraform`.

ГОВОРЮ

«Теперь покажу код, который описывает эквивалентный пяти-VM стенд. Для дальнейших лабораторий остаются машины из урока; Terraform-стенд создаю временно, проверяю в UI и удаляю своим же state.

Здесь Terraform разложен как нормальный инфраструктурный проект.

`versions.tf` фиксирует Terraform и провайдер. `providers.tf` настраивает провайдер. `variables.tf` объявляет типизированные входы и запрещает открыть SSH всему интернету. `locals.tf` описывает ровно пять ролей: node1–node3 для Kubernetes, lb1–lb2 для внешнего входа. `network.tf` и `security_groups.tf` отвечают за сеть. `compute.tf` вызывает модуль. В `modules/k8s_nodes` находятся переиспользуемые ресурсы VM, дисков и интерфейсов. `outputs.tf` не просто печатает IP: он формирует два inventory — для Kubespray и для внешнего HA.

`terraform.tfvars.example` безопасно коммитить как образец. Реальные CIDR и SSH-ключ идут в локальный `terraform.tfvars`. State, приватные ключи и созданные inventory Git игнорирует».

ПОКАЗАТЬ

Идти в указанном порядке:

- `versions.tf`
- `providers.tf`
- `variables.tf`

- `locals.tf`
- `network.tf` и `security_groups.tf`
- `compute.tf`
- `modules/k8s_nodes/main.tf`
- `outputs.tf`
- `backend.s3.tf.example`

📖 V2-C1-S06-C01 · Проверить production-shaped Terraform до apply

```
cd "$REPO_ROOT/infra/self-managed/terraform"
test -f terraform.tfvars || cp terraform.tfvars.example terraform.tfvars
terraform fmt -check -recursive
terraform init
terraform validate
terraform plan -out=/tmp/video2-self-managed.tfplan >/tmp/video2-self-managed-
plan.log
terraform show -json /tmp/video2-self-managed.tfplan \
  | jq -r '.resource_changes[] | [.address, (.change.actions | join(", "))] |
@tsv'
```

✅ После команды

Ожидаю: план ровно пяти VM — `node1`, `node2`, `node3`, `lb1`, `lb2` — плюс сеть, подсеть и правила доступа. В `terraform.tfvars` уже должны быть твой public IP `/32` и SSH public key.

🗣 ГОВОРЮ

«Plan сохраняю и вывожу безопасной таблицей `address → action`: так вижу пять VM и зависимые ресурсы без folder ID и адресов. Apply использует именно этот сохранённый plan. После apply Terraform отдаёт публичные адреса для SSH, private

addresses для межмашинного трафика и умеет собрать inventory, но в кадре я заменяю адреса и instance ID на `<redacted>`. Так выглядит связка Terraform → Ansible в инфраструктуре, которую контролирует наша команда.

После apply переключаюсь в Yandex Cloud и проверяю, что код действительно создал: сеть, подсеть, security group и пять VM. В UI ничего не исправляю».

📄 V2-C1-S06-C02 · Временно создать пять VM и проверить Terraform outputs

```
terraform apply -no-color /tmp/video2-self-managed.tfplan \  
  >/tmp/video2-self-managed-apply.log  
grep -E '^(Apply complete|Outputs:|[[[:alnum:]]_]+ = <sensitive>)' \  
  /tmp/video2-self-managed-apply.log  
terraform output -json nodes | jq 'with_entries(.value |= {  
  roles,  
  external_ip: "<redacted>",  
  internal_ip: "<redacted>",  
  instance_id: "<redacted>"  
})'  
terraform output -raw kubespray_inventory \  
  | sed -E 's/(ansible_host|ip|access_ip)=[^ ]+/\1=<redacted>/g'  
terraform output -raw external_ha_inventory \  
  | sed -E 's/(ansible_host|private_ip)=[^ ]+/\1=<redacted>/g'  
yc compute instance list --folder-id "$YC_FOLDER_ID" --format json \  
  | jq -r '.[ ] | [.name, .status] | @tsv'
```

✅ После команды

Ожидаю: временные `node1-node3` и `lb1-lb2`, сеть и два inventory как outputs. В кадре видны роли и структура, а IP/instance ID заменены на `<redacted>`. Рабочим стендом записи остаются пять VM из урока BoostMentor.

✅ БРАУЗЕРНАЯ ПРОВЕРКА

- Compute Cloud → пять VM Running.
- VPC → нужна сеть/подсеть.
- У каждой VM private IP; public IP используется для SSH.
- Не показывать project/folder IDs крупным планом.

🗣 ГОВОРЮ

«Временный стенд задачу выполнил. Сначала читаю destroy plan, затем удаляю только ресурсы этого state. Учебные VM из BoostMentor — другой жизненный цикл, они продолжают работать».

📄 V2-C1-S06-C03 · Удалить только временный Terraform-стенд

```
terraform plan -destroy -out=/tmp/video2-self-managed-destroy.tfplan \  
  >/tmp/video2-self-managed-destroy-plan.log  
terraform show -json /tmp/video2-self-managed-destroy.tfplan \  
  | jq -r '.resource_changes[] | [.address, (.change.actions | join(","))] |  
@tsv'  
terraform apply -no-color /tmp/video2-self-managed-destroy.tfplan \  
  >/tmp/video2-self-managed-destroy.log  
grep -E '^Destroy complete' /tmp/video2-self-managed-destroy.log  
yc compute instance list --folder-id "$YC_FOLDER_ID" --format json \  
  | jq -r '.[ ] | [.name, .status] | @tsv'
```

✅ После команды

Ожидаю: Terraform удаляет только ресурсы этого state. Пять VM учебной платформы живут отдельно и остаются для Kubespray и внешнего HA.

⚠ ЕСЛИ APPLY ДОЛГО ИДЁТ

Сказать «АНТРАКТ», остановить REC, дождаться завершения. После возврата показать последние строки apply и outputs. Не заполнять ожидание импровизацией.

ПЕРЕХОД

«Self-managed машины уже дала учебная лаборатория, а Terraform показал их production-shaped описание. Рядом создадим managed: Terraform похож, но граница ответственности другая».

c1.4 Managed Kubernetes через отдельный Terraform root

Время: 78:00–93:00 плюс ожидание apply

ЭКРАН: `infra/managed/terraform` и Yandex Cloud.

ГОВОРЮ

«Managed-кластер — отдельный Terraform root и отдельный state. Здесь есть сеть, service accounts, кластер и node group. Control plane создаёт и обслуживает провайдер, workers остаются видимыми ресурсами.

Открываю файлы по порядку: `main.tf` с версиями, `network.tf`, `service-accounts.tf`, `kubernetes.tf` и `outputs.tf`. Сетевые диапазоны у managed отдельные, чтобы стенды не пересекались.

В `private.auto.tfvars` лежат локальные CIDR и разрешённый адрес `/32`. Этот файл не коммитим. Plan сверяю до apply».

ПОКАЗАТЬ

- `main.tf`: provider/version/variables.
- `network.tf`: VPC, subnet, security group.
- `service-accounts.tf`.
- `kubernetes.tf`: cluster и node group.
- `outputs.tf`: CIDR и команда получения credentials.

V2-C1-S07-C01 · Managed-кластер тем же IaC-подходом

```
cd "$REPO_ROOT/infra/managed/terraform"
test -f private.auto.tfvars || cp private.auto.tfvars.example private.auto.tfvars
terraform fmt -check -recursive
terraform init
terraform validate
terraform plan -out=/tmp/video2-managed.tfplan >/tmp/video2-managed-plan.log
terraform show -json /tmp/video2-managed.tfplan \
  | jq -r '.resource_changes[] | [.address, (.change.actions | join(","))] | @tsv'
terraform apply -no-color /tmp/video2-managed.tfplan >/tmp/video2-managed-apply.log
grep -E '^(Apply complete|Outputs:|[:alnum:])+ = <sensitive>' \
  /tmp/video2-managed-apply.log
yc managed-kubernetes cluster list --format json \
  | jq -r '.[.] | [.name, .status] | @tsv'
yc managed-kubernetes node-group list --format json \
  | jq -r '.[.] | [.name, .status] | @tsv'
terraform output cidrs
```

После команды

Ожидая: managed control plane и node group. В браузере проверяем те же ресурсы в Yandex Cloud, не редактируем их руками.

БРАУЗЕРНАЯ ПРОВЕРКА

- Managed Service for Kubernetes → Cluster status Running.
- Node group status Ready.
- VPC/subnet с ожидаемым CIDR.
- Service accounts.

ЧТО ЭТО ЗНАЧИТ

Terraform используется и для managed, и для self-managed. Отличается объект управления: в одном случае Terraform создаёт обычные VM, а Kubernetes устанавливаем мы; в другом он вызывает API провайдера, который управляет control plane.

ПЕРЕХОД

«Managed пока достраивается. Возвращаюсь к трём обычным Linux-машинам и проверяю их до Kubespray».

Время: 93:00–108:00

ЭКРАН: `kubespray/inventory/video2/inventory.example.ini`, безопасный `ansible-inventory --graph` и терминал.

ГОВОРЮ

«Пять VM уже созданы одной лабораторией. После запуска я вне кадра вписал их SSH-адреса и путь к скачанному ключу в локальный env. Теперь прямым here-doc собираю два разных inventory: три Kubernetes-ноды для Kubespray и все пять машин для внешнего HA. Никакого скрытого генератора здесь нет.

В Kubernetes inventory node1 и node2 входят в `kube_control_plane`. Все три — в `etcd`. node2 и node3 — workers. node1 workload не принимает: так на маленьком стенде у нас два API-сервера, три члена etcd и два места для приложения.

`ansible_host` — адрес, по которому мой ноутбук подключается по SSH. `ip` и `access_ip` здесь не задаю: сначала Ansible снимает реальный private IPv4 из facts, и этот адрес используется для трафика между VM. Floating IP может вообще не быть назначен интерфейсу внутри VM.

На экране открываю только committed example и graph без hostvars. Реальный inventory нужен Ansible для подключения, но пять floating-адресов в запись не вывожу.

Для локального `admin.conf` включён `kubeconfig_localhost_ansible_host`: endpoint в артефакте будет указывать на `ansible_host` node1. Public SAN нужен, чтобы сертификат этот адрес принимал. Это лабораторный single-endpoint доступ, не HA endpoint для production.

Затем проверяю не только ping, но и sudo. Если доступ или become не работают, бессмысленно запускать двухчасовой playbook».

V2-C1-S08-C01 · Прочитать роли нод и проверить SSH/sudo

```
cd "$REPO_ROOT"
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
INV="$KUBESPRAY_ROOT/inventory/video2/inventory.ini"
source "$REPO_ROOT/recording/.recording.env"
SSH_KEY="$VIDEO2_SSH_KEY"
SSH_USER="${VIDEO2_SSH_USER:-root}"
: "${NODE1_SSH_HOST:?fill recording/.recording.env}"
: "${NODE2_SSH_HOST:?fill recording/.recording.env}"
: "${NODE3_SSH_HOST:?fill recording/.recording.env}"
: "${LB1_SSH_HOST:?fill recording/.recording.env}"
: "${LB2_SSH_HOST:?fill recording/.recording.env}"
install -d -m 0700 "$KUBESPRAY_ROOT/inventory/video2/group_vars/all"
cat > "$INV" <<EOF
[all]
node1 ansible_host=$NODE1_SSH_HOST ansible_user=$SSH_USER
node2 ansible_host=$NODE2_SSH_HOST ansible_user=$SSH_USER
node3 ansible_host=$NODE3_SSH_HOST ansible_user=$SSH_USER

[kube_control_plane]
node1
node2

[etcd]
node1
node2
node3

[kube_node]
node2
node3

[k8s_cluster:children]
```

```
kube_control_plane
kube_node
EOF
cat > "$KUBESPRAY_ROOT/inventory/video2/group_vars/all/all.yml" <<EOF
---
supplementary_addresses_in_ssl_keys:
  - $NODE1_SSH_HOST
EOF
cat > "$REP0_ROOT/ansible/external-ha/inventory.ini" <<EOF
[load_balancers]
lb1 ansible_host=$LB1_SSH_HOST ansible_user=$SSH_USER ha_overlay_ip=10.77.0.11
ha_priority=150
lb2 ansible_host=$LB2_SSH_HOST ansible_user=$SSH_USER ha_overlay_ip=10.77.0.12
ha_priority=100

[kubernetes_nodes]
node1 ansible_host=$NODE1_SSH_HOST ansible_user=$SSH_USER
node2 ansible_host=$NODE2_SSH_HOST ansible_user=$SSH_USER
node3 ansible_host=$NODE3_SSH_HOST ansible_user=$SSH_USER

[all:vars]
ansible_python_interpreter=/usr/bin/python3
ha_vip=$HA_VIP
ha_vip_prefix=24
ha_frontend_port=$HA_PORT
k8s_nodeport=$K8S_NODEPORT
EOF
chmod 600 "$SSH_KEY"
source "$KUBESPRAY_ROOT/.venv/bin/activate"
sed -n '1,160p' "$KUBESPRAY_ROOT/inventory/video2/inventory.example.ini"
ansible-inventory -i "$INV" --graph
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.ping
```

```
ansible -i "$INV" all --private-key "$SSH_KEY" --become -m  
ansible.builtin.command -a whoami
```

✓ После команды

Ожидаю: структура видна по безопасному example и graph без hostvars; два реальных inventory построены из пяти адресов одной учебной лаборатории, но адреса не печатаются. node1/node2 в control plane, все три в etcd, node2/node3 в workers; все отвечают `pong` и `root` через become.

ГОВОРЮ

«Теперь три причины типичного падения установки: включённый swap, неправильный cgroup и отсутствующий маршрут. Последней командой Ansible сам собирает private IPv4 facts. Локальный успех здесь ещё не гарантирует успех в CI: runner может идти с другого адреса, не иметь route или нужного ключа. Поэтому фиксируем, откуда идёт запуск».

☰ V2-C1-S09-C01 · Проверить Linux-предпосылки и private networking

```
ansible -i "$INV" all --private-key "$SSH_KEY" --become -m ansible.builtin.shell \
-a 'test -z "$(swapon --show --noheadings)"; test "$(stat -fc %T  
/sys/fs/cgroup)" = cgroup2fs; ip -4 route show default'
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.setup \
-a 'filter=ansible_default_ipv4'
```

✓ После команды

Ожидаю: swap пуст, cgroup v2, default route есть; `ansible_default_ipv4.address` — private IP. Public/floating IP применяется только для SSH.

⚠ ЕСЛИ PING НЕ ПРОШЁЛ

Проверить по порядку: VM Running → public IP → security group /32 → имя пользователя → ключ → route/VPN. Не отключать VPN и не менять системную сеть. Пустые логи Kubernetes здесь нормальны: Kubernetes ещё не установлен.

ПЕРЕХОД

«Доступ есть, роли назначены, Linux готов. Теперь открываю параметры, которые Kubespray применит, и только после этого запускаю установку».

Время: 108:00–131:00 плюс 30–60 минут ожидания

ЭКРАН: upstream Kubespray, наши group_vars, терминал.

ГОВОРЮ

«Перед командой читаю два слоя.

В upstream `cluster.yml` — вход в проект и импорты playbooks. В нашем `k8s-cluster.yml` — решения конкретного кластера: Kubernetes `v1.34.7`, containerd, Calico, Pod CIDR, Service CIDR и `kube_proxy_strict_arp: true` для MetalLB.

Inventory отвечает, где живёт компонент. Group vars отвечают, с какими параметрами его ставить. Roles выполняют работу».

V2-C1-S09-C02 · Перед запуском прочитать playbook и параметры кластера

```
UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
sed -n '1,220p' "$UPSTREAM/cluster.yml"
sed -n '1,220p' "$UPSTREAM/playbooks/cluster.yml"
sed -n '1,220p' "$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/k8s-cluster.yml"
find "$UPSTREAM/roles" -mindepth 1 -maxdepth 1 -type d -print | sort | sed -n '1,24p'
```

После команды

Ожидая: видны `kube_version`, containerd, Calico, Pod/Service CIDR, strictARP, `kubeconfig_localhost` и `kubeconfig_localhost_ansible_host`. Последняя настройка заставит локальный `admin.conf` использовать SSH-адрес node1, а не недоступный с ноутбука private IP. Никакой установки ещё не было.

ГОВОРЮ

«Главный запуск — обычный `ansible-playbook` на upstream `cluster.yml`. Не прячу его в `deploy.sh`. Мы видим задачи runtime, etcd, control plane, kubelet и CNI.

Процесс долгий. Я не буду читать полчаса бегущие строки. Отмечаю начало, говорю «АНТРАКТ», выключаю запись и возвращаюсь на `PLAY RECAP`. Если есть `failed`, это отдельный troubleshooting-дубль, а не повод делать вид, что всё готово.

После успешного реска Kubespray оставляет admin kubeconfig с настоящим CA. Я добавляю его в общий config под понятным контекстом `kubespray`. TLS-проверку не отключаю».

V2-C1-S09-C03 · Прямой запуск Kubespray и безопасный kubeconfig

```
PLAYBOOK="$KUBESPRAY_ROOT/.venv/bin/ansible-playbook"
pushd "$UPSTREAM" >/dev/null
"$PLAYBOOK" -i "$INV" cluster.yml --become --private-key "$SSH_KEY"
popd >/dev/null
KUBECONFIG_FILE="$KUBESPRAY_ROOT/inventory/video2/artifacts/admin.conf"
OLD_CONTEXT="$(KUBECONFIG="$KUBECONFIG_FILE" kubectl config current-context)"
KUBECONFIG="$KUBECONFIG_FILE" kubectl config rename-context "$OLD_CONTEXT"
kubespray
mkdir -p "$HOME/.kube"
KUBECONFIG="$HOME/.kube/config:$KUBECONFIG_FILE" kubectl config view --flatten >
/tmp/video2-kubeconfig
install -m 0600 /tmp/video2-kubeconfig "$HOME/.kube/config"
kubectl --context kubespray get nodes -o wide
```

После команды

Ожидая: `PLAY RECAP` без failed, затем три Ready-ноды. TLS не отключаем и `--insecure-skip-tls-verify` не используем.

✅ ПОКАЗАТЬ ПОСЛЕ ОЖИДАНИЯ

- Последние задачи и `PLAY RECAP`.
- `failed=0`.
- Три Ready-ноды и их версии/roles.
- Не показывать содержимое client-key-data.

🧠 ЧТО ФИЗИЧЕСКИ ПОЯВИЛОСЬ

На node1/node2 — API server и control-plane компоненты; на всех трёх — etcd; на node2/node3 — kubelet для workload; на каждой нужной ноде — containerd и Calico. До этой команды были просто Linux-VM.

ПЕРЕХОД

«Ready в `get nodes` — необходимая, но слабая проверка. Теперь докажем API, системные компоненты, сеть и реальный HTTP».

Время: 131:00–142:00

ЭКРАН: терминал → `kubernetes/devops-may-cry/base`.

ГОВОРЮ

«Проверка идёт слоями. Сначала ноды. Затем endpoint `/readyz` самого API server. Потом системные Pod'ы, DaemonSet Calico и Deployment CoreDNS.

После инфраструктуры ставлю наше приложение из base Kustomize. В манифесте нет кода Python внутри YAML. Это отдельный immutable multi-arch образ Go-приложения, security context, probes, requests/limits, PDB и spread по hostname».

ПОКАЗАТЬ

- `kubernetes/devops-may-cry/base/deployment.yaml`.
- image digest, не плавающий latest.
- startup/liveness/readiness probes.
- `runAsNonRoot`, read-only filesystem, drop capabilities.
- topology spread и PDB.

V2-C1-S10-C01 · Доказать, что кластер действительно работает

```
kubectl --context kubespray get nodes -o wide
kubectl --context kubespray get --raw='/readyz?verbose' | tail -12
kubectl --context kubespray -n kube-system get pods -o wide
kubectl --context kubespray -n kube-system get daemonset calico-node
kubectl --context kubespray -n kube-system get deployment coredns
kubectl --context kubespray apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context kubespray -n traffic-lab rollout status deployment/devops-may-cry --timeout=180s
kubectl --context kubespray -n traffic-lab get pods,service,endpointslice -o wide
```


✅ После команды

Ожидаю: API ready, системные Pod'ы Running, Calico на каждой ноде, CoreDNS доступен, приложение разложено по workers и EndpointSlice содержит ready endpoints.

🗣 ГОВОРЮ

«ClusterIP с ноутбука напрямую недоступен, поэтому создаю одноразовый curl-клиент внутри кластера. Он резолвит имя Service и вызывает readiness endpoint. После доказательства удаляю клиент».

📄 V2-C1-S10-C02 · Проверить Service из временного клиента

```
kubectl --context kubespray -n traffic-lab run verify-client \
  --image=curlimages/curl:8.12.1 --restart=Never --command -- sleep 3600
kubectl --context kubespray -n traffic-lab wait --for=condition=Ready pod/verify-
client --timeout=120s
kubectl --context kubespray -n traffic-lab exec verify-client -- curl -fsS
http://devops-may-cry/readyz
kubectl --context kubespray -n traffic-lab delete pod verify-client --wait=true
```

✅ После команды

Ожидаю: HTTP JSON со `status=ok`, затем одноразовый клиент удалён.

🧠 ЧТО ЭТО ДОКАЗЫВАЕТ

`get nodes` доказывает регистрацию kubelet. `/readyz` — состояние API server. System pods — базовые addons. Rollout + EndpointSlice + HTTP 200 — что scheduler, runtime, CNI, DNS и Service вместе довели запрос до приложения.

🚧 В ПРОДЕ

Если Pod не стартовал и application logs пусты, сначала смотрят Events, image pull, admission и scheduling. Ошибка registry auth или недоступный external service происходит до запуска процесса, поэтому пустой stdout не оправдывает гипотезу «сломался код приложения».

ПЕРЕХОД

«Кластер работает. Теперь на три минуты отступим назад и разберём, откуда вообще берутся маски и диапазоны. Без этого Pod CIDR выглядит как набор случайных цифр».

Время: 142:00–154:00

ЭКРАН: доска «Маска подсети: как считать»; при наличии — «От сигнала к биту».

ГОВОРЮ

«Начнём с самого низа. Транзистор можно использовать как быстрый переключатель: условно есть сигнал — один, нет сигнала — ноль. Один бит даёт два состояния. Два бита — четыре: 00, 01, 10, 11. Каждый следующий бит удваивает число комбинаций. Поэтому для n бит получаем два в степени n .

IPv4-адрес — 32 бита, которые для удобства записали четырьмя десятичными октетами. В каждом октете 8 бит, поэтому значений 256 — от нуля до 255.

Префикс говорит, сколько первых бит относится к сети. Для `/26` сетевых бит 26, свободных — 6. Два в шестой степени — 64 адреса. В сети `192.168.1.70/26` адрес .70 лежит в блоке .64–.127. .64 — адрес сети, .127 — broadcast, обычные хосты — .65–.126.

В Kubernetes отдельно выбираем subnet под, Pod CIDR и Service CIDR. Они не должны пересекаться друг с другом, VPC, офисной сетью и VPN. Потом менять их тяжело, поэтому это решение принимают до установки».

V2-C1-S11-C01 · Посчитать сеть на конкретном IPv4-примере

```
python3 - <<'PY'
import ipaddress

for value in ("192.168.1.70/26", "10.233.64.0/18", "10.233.0.0/18"):
    net = ipaddress.ip_network(value, strict=False)
    print(value)
    print("  network    ", net.network_address)
    print("  netmask    ", net.netmask)
    print("  broadcast  ", net.broadcast_address)
    print("  addresses  ", net.num_addresses)
    print("  first/last", net.network_address + 1, net.broadcast_address - 1)
PY
```

✅ После команды

Ожидая для `192.168.1.70/26`: сеть `192.168.1.64`, usable `.65–.126`, broadcast `.127`, всего 64 адреса.

🧠 ЧТО ЭТО ЗНАЧИТ

Маска — не случайный набор чисел вроде `255`. Это двоичная граница между сетевой и хостовой частью. `2^(32-prefix)` даёт размер блока; usable host range для обычной IPv4 subnet часто исключает network и broadcast, но Kubernetes и облака могут иметь свои правила резервирования.

⚠️ НЕ УПРОЩАТЬ ДО ОШИБКИ

Не говорить, что любой `/24` всегда даёт ровно 254 доступных адреса приложению: облачный провайдер может резервировать дополнительные IP, а Pod networking работает по своей модели.

ПЕРЕХОД

«Теперь посмотрим, как это проявилось в только что созданном кластере».

Время: 154:00–171:00

ЭКРАН: сначала доска «Calico vs Cilium», затем «Как работает Calico», затем терминал.

ГОВОРЮ

«Calico и Cilium решают базовую задачу CNI, но делают это разными dataplane и наборами возможностей. На нашей доске сравнение упрощённое: Calico здесь — привычный выбор с NetworkPolicy и маршрутизацией, Cilium — eBPF-native, Hubble и более глубокая L7-наблюдаемость. Cilium может заменить kube-proxy в соответствующей конфигурации, но это не автоматическое свойство любого кластера.

Наш стенд установлен с Calico. У CNI три базовых действия: подключить Pod к сети, назначить IP и обеспечить маршрут до других Pod'ов.

Внутри Pod виден `eth0`. На ноде ему соответствует veth-интерфейс. Calico IPAM выдаёт адреса из pool блоками, закреплёнными за нодами. Между нодами трафик идёт по маршруту и, в зависимости от конфигурации, напрямую, через IP-in-IP или VXLAN.

Сначала отделяю `Node.spec.podCIDR` от Calico IPAM block. На доске может быть пример блока `/26`, а на нашем кластере kube-controller-manager выдаёт нодам другие PodCIDR. Принцип тот же, но размер и сущность разные».

V2-C1-S12-C01 · Связать Node PodCIDR с реальными Pod IP

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
LAB_DIR="$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/09_1.7_cidr_to_node"
INV="$REPO_ROOT/kubespray/inventory/video2/inventory.ini"
SSH_KEY="${VIDEO2_SSH_KEY:-$HOME/.ssh/id_ed25519}"
kubectl --context kubespray get nodes \
  -o custom-columns='NODE:.metadata.name,INTERNAL-IP:.status.addresses[?
(@.type=="InternalIP")].address,POD-CIDR:.spec.podCIDR'
kubectl --context kubespray get ippools.crd.projectcalico.org -o yaml | \
  grep -E 'cidr:|blockSize:|ipipMode:|vxlanMode:'
kubectl --context kubespray get blockaffinities.crd.projectcalico.org \
  -o custom-columns='NODE:.spec.node,CIDR:.spec.cidr,STATE:.spec.state'
```

После команды

Ожидаю: три разных `Node.spec.podCIDR` и отдельно Calico pool/blockSize. Это связанные, но не одинаковые сущности.

ГОВОРЮ

«Теперь ставлю два диагностических Pod'а на разные worker-ноды. Показываю IP, `eth0` внутри первого Pod, ping до второго и хостовый маршрут. Если `vxlan.calico` отсутствует, я не объявляю VXLAN сломанным: сначала смотрю фактический режим IPPool. Команда специально допускает другой dataplane».

V2-C1-S12-C02 · Посмотреть veth, маршрут и overlay на живых Pod'ах

```
kubectl --context kubespray apply -f "$LAB_DIR/calico-proof.yaml"
kubectl --context kubespray -n network-proof wait --for=condition=Ready pod --all
--timeout=120s
kubectl --context kubespray -n network-proof get pods -o wide
POD_B_IP="$(kubectl --context kubespray -n network-proof get pod calico-b -o
jsonpath='{.status.podIP}')"
kubectl --context kubespray -n network-proof exec calico-a -- ip address show
eth0
kubectl --context kubespray -n network-proof exec calico-a -- ping -c 3
"$POD_B_IP"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
ansible -i "$INV" node2 --private-key "$SSH_KEY" --become -m
ansible.builtin.shell \
    -a "ip route get $POD_B_IP; ip -o link | grep cali | head; ip -d link show
vxlan.calico || true"
kubectl --context kubespray delete namespace network-proof --wait=true
```

После команды

Ожидая: Pod IP, `eth0` внутри Pod, `cali...`-интерфейсы/route на хосте и успешный ping между нодами. VXLAN показываем только если этот dataplane реально активен.

ЧТО ЭТО ДОКАЗЫВАЕТ

Связка «Pod IP → нода → veth → route → overlay/underlay» показана фактами живого кластера. Сравнительный слайд не заменяет эту проверку.

В ПРОДЕ

Менять CNI на работающем кластере дорого и рискованно. Обычно выбирают dataplane, MTU, encapsulation и policy model до создания workload. Для troubleshooting нужны не только `kubectl get pods`, но и IPPool, routes, interfaces, packet capture и события.

ПЕРЕХОД

«Self-managed готов. Забираю доступ к managed и ставлю два кластера рядом».

Время: 171:00–177:00

ЭКРАН: терминал и доска «Managed vs Self-managed».

ГОВОРЮ

«Managed credentials выдаёт CLI провайдера. Self-managed kubernetes создал Kubespray с нашим CA и public SAN. Я называю контексты `yc-managed` и `kubespray`, чтобы команда всегда явно показывала целевой кластер.

Не использую `--insecure-skip-tls-verify`. Если сертификат не подходит адресу API, исправляют SAN/endpoint или способ доступа, а не выключают проверку».

V2-C1-S13-C01 · Получить managed kubernetes и назвать контексты

```
cd "$REPO_ROOT/infra/managed/terraform"
$(terraform output -raw get_credentials_command)
kubectl config rename-context "$(kubectl config current-context)" yc-managed
kubectl --context yc-managed get nodes -o wide
kubectl config get-contexts
```

После команды

Ожидая: `kubespray` и `yc-managed` существуют одновременно, а у managed видны только worker-ноды.

СЕКРЕТЫ В КАДРЕ

Не раскрывать `client-key-data`, токены, содержимое реального kubernetes и cloud IDs. Для объяснения достаточно имени контекста, cluster server можно замаскировать.

ПЕРЕХОД

«Перед сравнением сделаем то, что в эксплуатации self-managed неизбежно: обслужим ноду и реально сменим версию».

Время: 177:00–188:00

ЭКРАН: доска «Безопасное обновление» → терминал.

ГОВОРЮ

«Сначала отдельно показываю механику обслуживания. `cordon` запрещает новые Pod'ы на ноде. `drain` выселяет управляемые workload, уважает PDB и пропускает DaemonSet. `uncordon` возвращает ноду scheduler'у.

Это ещё не upgrade. Версия kubelet и control plane от этих команд не меняется».

V2-C1-S14-C01 · Maintenance drill: cordon, drain, PDB, uncordon

```
kubectl --context kubespray apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context kubespray -n traffic-lab rollout status deployment/devops-may-cry --timeout=180s
kubectl --context kubespray cordon node2
kubectl --context kubespray -n traffic-lab get pods -l app=devops-may-cry -o wide
kubectl --context kubespray drain node2 --ignore-daemonsets --delete-emptydir-data
kubectl --context kubespray -n traffic-lab get pods -l app=devops-may-cry -o wide
kubectl --context kubespray -n traffic-lab get pdb devops-may-cry
kubectl --context kubespray uncordon node2
```

После команды

Ожидая: node2 становится unschedulable, обычные Pod'ы переезжают с учётом PDB, DaemonSet не выселяется, затем node2 возвращается.

ЧТО ПРОВЕРИТЬ

Pod'ы приложения должны остаться доступны. Если PDB блокирует drain, не добавлять `--disable-eviction` ради красивого дубля: сначала объяснить, какое availability-ограничение мешает.

ПЕРЕХОД

«Теперь готовим именно смену версии: состояние до, snapshot etcd, diff конфигурации, непрерывная проба, upgrade playbook и доказательство после».

C1.8-A Настоящий upgrade 1.34.7 → 1.35.4

Время: 188:00–214:00 плюс ожидание

ЭКРАН: терминал, `kubespray/versions`, upstream `upgrade-cluster.yml`.

ГОВОРЮ

«Перед изменением снимаю факты: версии нод, готовность API и snapshot etcd. Snapshot не заменяет проверенный restore runbook, но без него начинать изменение control plane ещё хуже. Отдельно смотрю срок сертификатов».

V2-C1-S15-C01 · До upgrade: версии, API и snapshot etcd

```
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
INV="$KUBESPRAY_ROOT/inventory/video2/inventory.ini"
GROUP_VARS="$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/k8s-
cluster.yml"
SSH_KEY="{VIDE02_SSH_KEY:~$HOME/.ssh/id_ed25519}"
source "$KUBESPRAY_ROOT/.venv/bin/activate"
kubectl --context kubespray get nodes \
  -o custom-
columns='NODE:.metadata.name,READY:.status.conditions[-1].status,VERSION:.status.
nodeInfo.kubeletVersion'
kubectl --context kubespray get --raw='/readyz?verbose' | tail -12
ansible -i "$INV" node1 --private-key "$SSH_KEY" --become -m
ansible.builtin.shell -a \
  'install -d -m 0700 /var/backups/etcd; /usr/local/bin/etcdctl.sh --
endpoints=https://127.0.0.1:2379 snapshot save /var/backups/etcd/video2-pre-
1.35.4.db; /usr/local/bin/etcdutl snapshot status /var/backups/etcd/video2-pre-
1.35.4.db -w table'
ansible -i "$INV" node1 --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a '/usr/local/bin/kubeadm certs check-expiration'
```

✅ После команды

Ожидаю: baseline `v1.34.7`, API ready, snapshot со status table и срок сертификатов.

🗣 ГОВОРЮ

«Целевая версия заранее проверена в матрице именно этого Kubespray tag. Меняю одну строку через подготовленный version file и показываю diff. Не прыгаю через несколько minor-версий и не выдаю произвольный номер за поддерживаемый».

📄 V2-C1-S15-C02 · Показать единственную смену версии

```
cp "$KUBESPRAY_R00T/versions/k8s-cluster-1.35.4.yml" "$GROUP_VARS"  
git -C "$REPO_R00T" diff --  
kubespray/inventory/video2/group_vars/k8s_cluster/k8s-cluster.yml
```

✅ После команды

Ожидаю: содержательный diff `kube_version: 1.34.7 → 1.35.4`, а не россыпь несвязанных правок.

🗣 ГОВОРЮ

«Во втором окне оставляю port-forward, в третьем — curl раз в секунду. Это не обещание нулевого простоя. После прогона мы увидим наблюдаемое поведение и сможем отличить короткий разрыв от полной потери сервиса».

📄 V2-C1-S15-C03 · Непрерывная HTTP-проба во время upgrade

```
# Терминал 1:
kubectl --context kubespray -n traffic-lab port-forward service/devops-may-cry
18080:80 &
printf '%s\n' "$!" > /tmp/video2-upgrade-port-forward.pid
wait "$(cat /tmp/video2-upgrade-port-forward.pid)"
# Терминал 2:
while true; do date +%T; curl -fsS http://127.0.0.1:18080/readyz || echo FAIL;
sleep 1; done &
printf '%s\n' "$!" > /tmp/video2-upgrade-probe.pid
wait "$(cat /tmp/video2-upgrade-probe.pid)"
```

✅ После команды

Ожидаю: видимый поток `status=ok`. Это наблюдение, не обещание нулевого downtime.

🗣️ ГОВОРЮ

«Теперь запускаю upstream `upgrade-cluster.yml` напрямую. Kubespray обновляет компоненты по группам и нодам. Долгий вывод режу так же: начало → «АНТРАКТ» → возвращение на ресар».

📄 V2-C1-S15-C04 · Запустить настоящий upgrade playbook

```
PLAYBOOK="$KUBESPRAY_ROOT/.venv/bin/ansible-playbook"
pushd "$UPSTREAM" >/dev/null
"$PLAYBOOK" -i "$INV" upgrade-cluster.yml \
  --become --private-key "$SSH_KEY"
popd >/dev/null
```

✅ После команды

Ожидаю: последовательное обслуживание нод и `PLAY RECAP` без failed. Это уже upgrade, в отличие от одного `drain`.

ГОВОРЮ

«После resar не заканчиваю на слове changed. Снова проверяю версию каждой ноды, API ready, системные Pod'ы, rollout приложения и Warning events. Критерий завершения — сервис восстановлен и новая версия подтверждена. После проверки команда останавливает обе фоновые пробы и освобождает локальный порт».

V2-C1-S15-C05 · Доказать восстановление после upgrade

```
kubectl --context kubespray get nodes \
  -o custom-
columns='NODE:.metadata.name,READY:.status.conditions[-1].status,VERSION:.status.
nodeInfo.kubeletVersion'
kubectl --context kubespray get --raw='/readyz?verbose' | tail -12
kubectl --context kubespray -n kube-system get pods -o wide
kubectl --context kubespray -n traffic-lab rollout status deployment/devops-may-
cry --timeout=180s
kubectl --context kubespray -n traffic-lab get pods -o wide
kubectl --context kubespray get events -A --field-selector type=Warning --sort-
by=.lastTimestamp | tail -12
for pid_file in /tmp/video2-upgrade-probe.pid /tmp/video2-upgrade-port-
forward.pid; do
  if test -f "$pid_file"; then
    kill "$(cat "$pid_file")" 2>/dev/null || true
    rm -f "$pid_file"
  fi
done
```

После команды

Ожидаю: все ноды Ready на `v1.35.4`, API ready, workload доступен. Warning events читаем, а не скрываем. В конце обе upgrade-пробы остановлены, порт `18080` освобождён.

В ПРОДЕ

Перед upgrade дополнительно нужны release notes, deprecated APIs, backup/restore rehearsal, окно изменений, наблюдаемость и rollback/stop criteria. Safe workaround вроде временного preload image может спасти лабораторию, но не становится production-решением: долг фиксируют и закрывают нормальным registry/IAM.

ПЕРЕХОД

«Теперь есть что сравнивать: оба кластера живы, а self-managed ещё и прошёл обслуживание с upgrade».

Время: 214:00–228:00

ЭКРАН: доска с итоговой таблицей → split terminal/browser.

ГОВОРЮ

«Сравниваем по шести наблюдаемым признакам.

Первое — ноды. В self-managed видны control-plane и workers. В managed — только worker-ноды.

Второе — etcd. В self-managed его Pod'ы и backup — наша ответственность. В managed control plane скрыт.

Третье — сертификаты. На self-managed я сам смотрю срок и планирую ротацию. Managed обслуживает control-plane-сертификаты, но это не отменяет наших secrets и app certificates.

Четвёртое — внешний Service. В managed облачный контроллер получает адрес у провайдера. В self-managed сейчас увидим `pending` и установим MetalLB.

Пятое — обновление. Managed даёт API/операцию провайдера; self-managed требует playbook, sequencing и доказательства.

Шестое — цена ответственности. Managed оплачивается деньгами и ограничениями платформы. Self-managed даёт больше контроля, но требует времени, on-call, backup, patching и capacity».

V2-C1-S16-C01 · Сравнить владение control plane, etcd и внешним IP

```
kubectl --context yc-managed apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context yc-managed -n traffic-lab rollout status deploy/devops-may-cry
--timeout=180s
kubectl --context yc-managed -n traffic-lab patch svc devops-may-cry \
  --type merge -p '{"spec":{"type":"LoadBalancer"}}'
kubectl --context kubespray get nodes
kubectl --context yc-managed get nodes
kubectl --context kubespray -n kube-system get pods | grep etcd
kubectl --context yc-managed -n kube-system get pods | grep etcd || true
kubectl --context yc-managed -n traffic-lab wait \
  --for=jsonpath='{.status.loadBalancer.ingress[0].ip}' \
  service/devops-may-cry --timeout=300s
kubectl --context yc-managed -n traffic-lab get svc devops-may-cry -o wide
```

После команды

Ожидая: self-managed показывает control-plane/etcd, managed скрывает их; облачный Service получает внешний адрес от провайдера.

ГОВОРЮ

«И ещё доступ. У managed обычно короткоживущий токен через cloud identity. У self-managed часто начинают с клиентского сертификата; его утечка равна утечке доступа, поэтому kubeconfig защищаем как секрет. В зрелой схеме и self-managed может использовать OIDC или другой централизованный вход».

V2-C1-S16-C02 · Сравнить kubernetes и ответственность

```
kubectl config get-contexts
kubectl config view --context yc-managed --minify -o yaml
kubectl config view --context kubespray --minify -o yaml
stat -f '%Sp %N' "$HOME/.kube/config"
yc managed-kubernetes cluster list --format json \
  | jq -r '.[.] | [.name, .status] | @tsv'
yc managed-kubernetes node-group list --format json \
  | jq -r '.[.] | [.name, .status] | @tsv'
```

После команды

Ожидаю: два разных способа доступа; содержимое сертификатов/токенов в кадре не раскрываем. Managed снимает часть эксплуатации платформы, но не отвечает за workload.

ФОРМУЛИРОВКА

Не говорить «managed лучше» или «self-managed лучше». Вопрос — какую часть эксплуатации команда готова купить у провайдера и какую обязана контролировать сама.

ПЕРЕХОД

«Один пункт таблицы пока красный: LoadBalancer в self-managed. Чиним его руками и сразу проверяем границу L2 в облаке».

Время: 228:00–251:00

ЭКРАН: доска «L2 vs L3 в MetalLB» → VS Code → терминал.

ГОВОРЮ

«Service типа LoadBalancer — это запрос на внешний адрес. В managed его обрабатывает интеграция облака. В обычном self-managed кластере обработчика нет, поэтому адрес остаётся `pending`».

V2-C1-S17-C01 · До MetalLB: Service LoadBalancer остаётся pending

```
cd "$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/11_1.9_metallb"
kubectl --context kubespray apply -k ./devops-may-cry-base
kubectl --context kubespray -n traffic-lab rollout status deploy/devops-may-cry -
-timeout=180s
kubectl --context kubespray apply -f ./devops-may-cry-loadbalancer.yaml
kubectl --context kubespray -n traffic-lab get svc devops-may-cry-metallb
```

После команды

Ожидая: `EXTERNAL-IP <pending>`. В self-managed кластере нет cloud-controller, который сам выделит адрес.

ГОВОРЮ

«Для L2 режима MetalLB нужен реальный свободный pool в той же сети, где соседи могут получить ARP-анонс. Pool нельзя выбирать по принципу «ping молчит»: адрес может быть зарезервирован DHCP или другим IPAM.

Kube-proxy в IPVS-режиме требует strictARP. Мы включили его до установки кластера и сейчас проверяем live config».

V2-C1-S17-C02 · Проверить pool и strictARP до установки

```
source "$REPO_ROOT/recording/.recording.env"
: "${NODE_SUBNET_CIDR:?fill recording/.recording.env}"
: "${METALLB_POOL_START:?fill recording/.recording.env}"
: "${METALLB_POOL_END:?fill recording/.recording.env}"
printf 'node subnet: %s\nreserved pool: %s-%s\n' \
    "$NODE_SUBNET_CIDR" "$METALLB_POOL_START" "$METALLB_POOL_END"
kubectl --context kubespray get nodes -o wide
kubectl --context kubespray -n kube-system get configmap kube-proxy -o yaml |
grep -A3 -B3 strictARP
```

После команды

Ожидая: pool входит в node subnet, зарезервирован вне DHCP/IPAM, `strictARP: true`. Ping не доказывает, что IP свободен.

ГОВОРЮ

«Устанавливаю chart версии `0.16.1` напрямую. Затем применяю два CR: `IPAddressPool` и `L2Advertisement`. Speaker на нодах объявляет выбранный адрес в L2-домене».

ПОКАЗАТЬ

- `metallb-demo.yaml` с placeholders.
- `IPAddressPool`.
- `L2Advertisement`.
- В BGP reference: `BGPPeer` и `BGPAdvertisement`, но не применять.

V2-C1-S17-C03 · Установить MetalLB L2 и получить EXTERNAL-IP

```
helm repo add metallb https://metallb.github.io/metallb
helm repo update metallb
helm upgrade --install metallb metallb/metallb \
  --namespace metallb-system --create-namespace \
  --version 0.16.1 --wait --timeout 5m
METALLB_CONFIG="$(mktemp "${TMPDIR:-/tmp}/video2-metallb.XXXXXX.yaml")"
sed -e "s/__POOL_START__/$METALLB_POOL_START/g" \
  -e "s/__POOL_END__/$METALLB_POOL_END/g" \
  ./metallb-demo.yaml > "$METALLB_CONFIG"
cat "$METALLB_CONFIG"
kubectl --context kubespray apply -f "$METALLB_CONFIG"
kubectl --context kubespray -n metallb-system get pods -o wide
kubectl --context kubespray -n metallb-system get ipaddresspool,l2advertisement -
o wide
kubectl --context kubespray -n traffic-lab wait \
  --for=jsonpath='{.status.loadBalancer.ingress[0].ip}' \
  service/devops-may-cry-metallb --timeout=120s
kubectl --context kubespray -n traffic-lab get svc devops-may-cry-metallb -o wide
kubectl --context kubespray -n metallb-system logs \
  -l app.kubernetes.io/component=speaker --tail=40
```

После команды

Ожидаю: controller/speaker Ready и Service получает адрес из pool. Это доказывает распределение адреса, но ещё не внешнюю достижимость.

ГОВОРЮ

«Service получил EXTERNAL-IP. Это доказательство назначения адреса, но не доказательство, что клиент снаружи до него дошёл.

В физической сети или разрешённом L2-сегменте сосед увидит ARP и curl пройдёт. Многие cloud VPC фильтруют чужие ARP-анонсы или не дают VM владеть произвольным адресом. Тогда статус заполнен, а трафика нет. Это не полурбочий LoadBalancer — это неподходящая модель сети.

L3/BGP решает задачу иначе: MetalLB анонсирует маршруты настоящему router peer. Без маршрутизатора и настроенного peering применять BGP YAML бессмысленно, поэтому показываю его как reference, а не разыгрываю успех».

V2-C1-S17-C04 · Проверить границу L2 в облачной сети и показать BGP reference

```
METALLB_IP="$(kubectl --context kubespray -n traffic-lab get service devops-may-cry-metallb -o jsonpath='{.status.loadBalancer.ingress[0].ip}')"
INV="$REPO_ROOT/kubespray/inventory/video2/inventory.ini"
SSH_KEY="${VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
ansible -i "$INV" node1 --private-key "$SSH_KEY" -m ansible.builtin.shell \
    -a "ip neigh show $METALLB_IP; curl --max-time 5 -sv http://$METALLB_IP/readyz;
rc=\$?; printf 'curl_rc=%s\n' \"\$rc\"; exit 0"
sed -n '1,220p' ./metallb-bgp-reference.yaml
rm -f "$METALLB_CONFIG"
```

После команды

Ожидая одно из двух: HTTP 200 и `curl_rc=0` в разрешённом L2-домене либо контролируемый timeout с ненулевым `curl_rc` в cloud VPC, которая не пропускает нужный ARP/L2. Ansible-задача остаётся зелёной, потому что обе ветки здесь являются измерением. BGP-файл не применяем без реального peer/router.

L2 И BGP КОРОТКО

- L2: один speaker отвечает за IP через ARP/NDP; проще, но ограничен broadcast domain.

- BGP: ноды анонсируют prefix маршрутизатору; сложнее, требует сетевой инфраструктуры, зато лучше масштабируется и может использовать ECMP.
- В облаке часто берут native LB. В on-prem/bare metal MetalLB может быть нормальным решением, если сетевая модель это поддерживает.

ПЕРЕХОД

«MetalLB живёт внутри Kubernetes и не заменяет отдельный HA-этаж перед кластером. Теперь покажу внешний вход на двух Linux-балансировщиках».

Время: 251:00–282:00

ЭКРАН: доска «Внешняя точка входа» → `ansible/external-ha` → два терминала.

ГОВОРЮ

«Схема другая: клиент обращается к VIP, keepalived переносит его между lb1 и lb2, HAProxy в L4 режиме отправляет TCP на NodePort каждой Kubernetes-ноды, а дальше работает kube-proxy и Service.

В этой лаборатории VIP `10.77.0.10` живёт внутри VXLAN между двумя LB. Это позволяет воспроизвести VRRP-подобный failover в облаке, но не превращает VIP в публичный cloud IP. Floating addresses нужны только для SSH.

Сначала публикую наше приложение NodePort `30080` и проверяю EndpointSlice. Потом читаю committed inventory example и graph без hostvars; реальный inventory с floating-адресами на экран не вывожу. Ansible сам снимает private IP facts, их не переписываем с панели».

📄 V2-C1-S18-C01 · Подготовить NodePort и прочитать HA inventory

```
cd "$REPO_ROOT"
HA_DIR="$REPO_ROOT/ansible/external-ha"
INV="$HA_DIR/inventory.ini"
SSH_KEY="${VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
source "$REPO_ROOT/recording/.recording.env"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
kubectl --context kubespray apply -f "$REPO_ROOT/kubernetes/devops-may-
cry/services/nodeport.yaml"
kubectl --context kubespray -n traffic-lab get service devops-may-cry-ha-nodeport
kubectl --context kubespray -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-ha-nodeport -o wide
sed -n '1,180p' "$HA_DIR/inventory.example.ini"
ansible-inventory -i "$INV" --graph
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.ping
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.setup \
  -a 'filter=ansible_default_ipv4'
```

✅ После команды

Ожидаю: NodePort `30080`, ready endpoints, безопасный example и graph с пятью host names без hostvars. Реальный inventory адреса не печатает; floating IP нужен только для SSH, private facts используются для VXLAN/backends.

🗣️ ГОВОРЮ

«Открываю `site.yml`. Первый play собирает facts со всех пяти VM. Роль `vxlan_l2` создаёт общий лабораторный L2 для lb1/lb2. Роль `haproxy` ставит L4 frontend и три private NodePort backend. Роль `keepalived` следит за HAProxy и назначает VIP только живому балансировщику.

В шаблоне HAProxy `mode tcp`. Проверка backend — HTTP `GET /readyz`, но dataplane остаётся L4. В keepalived видны `interval/fall`, `advert_int` и `nopreempt`.

Сначала syntax check, затем отдельный preflight с lb1 до каждого private backend. Если backend недоступен, основной playbook не запускаю».

☰ V2-C1-S18-C02 · Прочитать роли и развернуть HAProxy/keepalived

```
sed -n '1,240p' "$HA_DIR/site.yml"
sed -n '1,220p' "$HA_DIR/roles/haproxy/templates/haproxy.cfg.j2"
sed -n '1,220p' "$HA_DIR/roles/keepalived/templates/keepalived.conf.j2"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --syntax-check --private-key "$SSH_KEY"
ansible-playbook -i "$INV" "$HA_DIR/preflight-backends.yml" --private-key "$SSH_KEY"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --private-key "$SSH_KEY"
```

✅ После команды

Ожидая: preflight достигает всех private NodePort backends; затем `PLAY RECAP` без failed. HAProxy работает в L4 `mode tcp`, health check идёт на `/readyz`.

🗣 ГОВОРЮ

«До отказа нужны три доказательства: оба сервиса active, VIP ровно на одном LB и HTTP 200 через VIP».

V2-C1-S18-C03 · Доказать VIP и HTTP до отказа

```
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
VIP_OWNERS=()
for host in lb1 lb2; do
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
    | grep -q "$HA_VIP/24"; then
    VIP_OWNERS+=("$host")
  fi
done
test "${#VIP_OWNERS[@]}" -eq 1
printf 'VIP owner: %s\n' "${VIP_OWNERS[0]}"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.uri \
  -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

После команды

Ожидая: оба сервиса active, VIP `10.77.0.10` находится ровно на одном LB, HTTP 200. Это лабораторный VIP внутри VXLAN-пары, не публичный cloud IP.

ГОВОРЮ

«Активный LB не зашит в сценарий: определяю владельца VIP по факту. На lb1 запускаю фоновую HTTP-пробу раз в секунду. Затем останавливаю только HAProxy на активном балансировщике.

Track script замечает отказ, keepalived снимает VIP, второй LB становится владельцем. Bounded-циклом жду сразу три условия: владелец снова ровно один, это уже другой LB и HTTP вернулся к 200. Затем показываю journal и probe log. Длительность

провала читаем с пробы; не обещаем красивую цифру заранее».

📄 V2-C1-S18-C04 · Найти активный LB и показать failover

```
VIP_OWNERS=()
for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
        -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' | grep -q
"$HA_VIP/24"; then
        VIP_OWNERS+=("$host")
    fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ACTIVE_LB="${VIP_OWNERS[0]}"
printf 'VIP owner before failure: %s\n' "$ACTIVE_LB"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.shell \
    -a "rm -f /tmp/vip-probe.log; nohup sh -c 'for i in \$(seq 1 25); do printf
\"%s \" \"\$(date +%T)\"; curl --max-time 2 -sS -o /dev/null -w \"%
{http_code}\\n\" http://$HA_VIP:$HA_PORT/readyz || echo timeout; sleep 1; done'
>/tmp/vip-probe.log 2>&1 </dev/null &"
ansible -i "$INV" "$ACTIVE_LB" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'systemctl stop haproxy'
NEW_OWNER=''
for _ in $(seq 1 30); do
    VIP_OWNERS=()
    for host in lb1 lb2; do
        if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
            -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
            | grep -q "$HA_VIP/24"; then
            VIP_OWNERS+=("$host")
        fi
    done
    if [ "${#VIP_OWNERS[@]}" -eq 1 ] \
        && [ "${VIP_OWNERS[0]}" != "$ACTIVE_LB" ] \
        && ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
```

```

    -m ansible.builtin.uri \
    -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200" \
    >/dev/null 2>&1; then
NEW_OWNER="${VIP_OWNERS[0]}"
break
fi
sleep 1
done
test -n "$NEW_OWNER"
printf 'VIP owner after failure: %s\n' "$NEW_OWNER"
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become -m
ansible.builtin.shell \
    -a "journalctl -u keepalived --since '3 minutes ago' --no-pager | tail -24"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'cat /tmp/vip-probe.log'
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.uri \
    -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"

```

✅ После команды

Ожидая: VIP переезжает на другой LB, в keepalived есть смена состояния, в probe log виден фактический разрыв и последующие 200. Не обещаем `<1s`.

🗣 ГОВОРЮ

«Возвращаю HAProxy. Из-за `nopreempt` восстановленный узел не отбирает VIP автоматически. Это нормально: важно, что оба сервиса active, один владелец VIP и HTTP 200».

V2-C1-S18-C05 · Restore и итоговая проверка

```
ansible -i "$INV" "$ACTIVE_LB" --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl start haproxy'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
VIP_OWNERS=()
for host in lb1 lb2; do
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
    | grep -q "$HA_VIP/24"; then
    VIP_OWNERS+=("$host")
  fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.uri \
  -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

После команды

Ожидая: оба сервиса active, владелец VIP по-прежнему ровно один и HTTP 200. Из-за `no-preempt` VIP после восстановления не обязан вернуться на lb1.

В ПРОДЕ

Здесь обязательно добавились бы реальный L2/BGP/cloud-failover механизм, firewall, observability, config delivery, secret management, несколько failure modes и регулярный failover test. DNS может переключать между endpoints, но TTL, caches и время реакции делают его другим механизмом, не мгновенной заменой VIP.

ЕСЛИ FAILOVER НЕ ПРОШЁЛ

Остановить дубль. Проверить facts/private IP, UDP 4789 для лабораторного VXLAN, интерфейс `vxlan100`, keepalived journal, HAProxy status и доступность NodePort с lb1. Не трогать системный VPN.

ПЕРЕХОД

«Теперь у нас два живых кластера, настоящий upgrade, MetalLB и отдельная отказоустойчивая точка входа. Соберу выводы».

Время: 282:00–291:00

ЭКРАН: итоговая таблица → карта следующих частей → DEVOPS MAY CRY.

ГОВОРЮ

«Начинали с двух файлов Terraform и локального Ansible. Рабочие пять VM получили из урока, такой же состав стенда описали в Terraform, а затем собрали Kubespray-кластер, managed-кластер и отдельную пару балансировщиков.

Главная цепочка такая. Terraform создаёт инфраструктуру и отдаёт outputs. Inventory назначает машинам роли. Ansible приводит Linux к нужному состоянию. Kubespray этими же механизмами ставит Kubernetes. После установки мы доказываем API, CNI, DNS и HTTP, а не останавливаемся на `get nodes`.

Managed и self-managed отличаются не наличием YAML и не качеством приложений. Отличается граница эксплуатационной ответственности.

MetalLB закрывает Service LoadBalancer внутри подходящей сетевой модели. Внешний HA-этаж решает другую задачу: стабильная точка входа и failover пары балансировщиков.

В следующей части DEVOPS MAY CRY перестанет быть просто тестовым Deployment. Разберём, как scheduler выбирает ноду, выделим место для Postgres, пройдём taints, affinity, topology spread, quota и Pending — всё на этом же приложении».

ПОКАЗАТЬ

- Список пяти VM и два Kubernetes context.
- Коротко `kubectl get nodes` на обоих.
- Финальная схема: Terraform → inventory → Kubespray → cluster → Service → external HA.

- Карта части 2 без обещания «через минуту», если запись будет отдельной.

ПОСЛЕ REC

📄 V2-C1-S19-C01 · Cleanup только после остановки записи

```
kubectl --context kubespray -n traffic-lab delete service devops-may-cry-metallb
--ignore-not-found
helm uninstall metallb -n metallb-system || true
kubectl --context kubespray delete namespace metallb-system --ignore-not-found
cd "$REPO_ROOT/infra/managed/terraform"
terraform destroy
cd "$REPO_ROOT/infra/self-managed/terraform"
terraform destroy
```

✅ После команды

Ожидая: cleanup выполняется только после REC и после сохранения доказательств. Перед каждым destroy читаем plan; state и inventory не показываем в кадре.

⚠️ ВАЖНО

Cleanup не читать и не выполнять в основном дубле. Если части 2–4 пишутся на тех же стендах, ничего не уничтожать. Сначала сохранить запись, логи и нужные доказательства.

- Хороший дубль: вслух «ИЗУМРУД».
- Долгое ожидание: «АНТРАКТ», после возврата «РУБЕЖ».
- Ошибка: «БРАК-БРАК», остановить, восстановить состояние, новый дубль начать с полного переходного предложения.
- Не импровизировать ожидаемый вывод. Сверить команду по ID и выполнить reset вне REC.
- Не использовать README как запасной суфлёр: если в этой колоде не объяснён файл или результат, это баг материала.

Время: 00:00–04:00

ЭКРАН: доска: агентство → диспетчерская → три Kubernetes-ноды.

ГОВОРЮ

«DEVOPS MAY CRY — наша учебная ночная диспетчерская. В первой части мы собрали для неё кластер и запустили Go-сервис. Теперь решим, где в этом кластере должны жить разные компоненты.

У приложения есть stateless-диспетчеры и PostgreSQL. Диспетчеры можно размножать и раскладывать по workers. Базу иногда приходится привязать к отдельному классу нод: другой диск, больше памяти, отдельная зона отказа. Kubernetes сам не угадает такое решение. Мы задаём его через taints, tolerations, affinity и topology spread.

В этой части я специально получу несколько Pending. Это не поломка стенда. Каждый раз сначала читаю Events, потом меняю ровно одно условие. Так видно, какое правило реально принял Scheduler».

ПОКАЗАТЬ

- `kubernetes/devops-may-cry/base`: две реплики stateless-приложения.
- `02_ЧАСТЬ_2_SCHEDULER/15_2.2_taint_toleration`: два варианта PostgreSQL.
- На доске: node1 — control plane; node2/node3 — workers.

ПЕРЕХОД

«Начну не с YAML из интернета, а со схемы, которую знает сам API Server».

Время: 04:00–11:00

ЭКРАН: терминал, затем `kubectl explain` рядом с доской Scheduler.

ГОВОРЮ

«`required` — обязательное условие. Если подходящей ноды нет, Pod остаётся Pending. `preferred` — вес в пользу подходящей ноды; Scheduler может выбрать другую, если идеального варианта нет.

Хвост `IgnoredDuringExecution` относится к уже запущенному Pod. Условие проверяется при планировании. Если потом label ноды изменился, kubelet не выселяет Pod только из-за этого. Через несколько минут это проверим на PostgreSQL».

V2-C2-S01-C01 · Зафиксировать self-managed context и спросить API о схеме affinity

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"
set -a
source recording/.recording.env
set +a
CTX="${SELF_CONTEXT:-kubespray}"
kubectl --context "$CTX" cluster-info
kubectl --context "$CTX" explain pod.spec.affinity
kubectl --context "$CTX" explain pod.spec.affinity.nodeAffinity
kubectl --context "$CTX" explain pod.spec.affinity.podAntiAffinity
```

✅ После команды

Ожидая: отвечает self-managed API; у nodeAffinity и podAntiAffinity видны

`requiredDuringSchedulingIgnoredDuringExecution` и

`preferredDuringSchedulingIgnoredDuringExecution`.

🧠 ЧТО ЭТО ЗНАЧИТ

`kubectl explain` берёт OpenAPI-схему из текущего API Server. Это надёжнее памяти и случайного блога: поля соответствуют версии нашего кластера.

ПЕРЕХОД

«Теперь зададим нодe две независимые характеристики: label говорит, что это database-нода; taint запрещает случайным Pod'ам на неё заходить».

c2.2 Taint отталкивает, toleration разрешает вход

Время: 11:00–26:00

ЭКРАН: `postgres-pending.yaml` → `postgres-scheduled.yaml` → терминал.

ГОВОРЮ

«На node3 ставлю label `tier=database` и taint `dedicated=database:NoSchedule`. Label сам никого не притягивает. Taint сам не выбирает нужный workload — он только не пускает Pod без пропуска.

Секрет создаю из терминала с выключенным echo. В Git пароль не хранится, на экране его тоже нет. Для настоящего production здесь был бы внешний secret manager или оператор, но механизм Scheduler от этого не меняется».

V2-C2-S02-C01 · Подготовить выделенную database-ноду и секрет без вывода пароля

```
LAB="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/15_2.2_taint_toleration"
kubectl --context "$CTX" create namespace traffic-lab --dry-run=client -o yaml |
kubectl --context "$CTX" apply -f -
kubectl --context "$CTX" describe node node1 | sed -n '/Taints:/p'
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier=database
--overwrite
kubectl --context "$CTX" taint node node3 dedicated=database:NoSchedule
read -r -s -p 'temporary DB password: ' DB_PASSWORD; echo
kubectl --context "$CTX" -n traffic-lab create secret generic devops-may-cry-db \
  --from-literal=POSTGRES_DB=devopsmaycry \
  --from-literal=POSTGRES_USER=devopsmaycry \
  --from-literal=POSTGRES_PASSWORD="$DB_PASSWORD" \
  --dry-run=client -o yaml | kubectl --context "$CTX" apply -f -
unset DB_PASSWORD
```


✓ После команды

Ожидая: node1 уже защищён control-plane taint; node3 получает label и `dedicated=database:NoSchedule`; Secret создан, пароль в терминал не выведен.

🖥 ПОКАЗАТЬ

Открыть `postgres-pending.yaml` и назвать четыре вещи:

- required nodeAffinity ищет label database;
- toleration отсутствует;
- образ PostgreSQL закреплён digest'ом;
- данные лежат в `emptyDir` — это одноразовая scheduler-лаба, не production storage.

🗣 ГОВОРЮ

«Этот манифест намеренно неполный. Он хочет node3 по affinity, но не умеет пройти её taint. Применяю и не гадаю по статусу — читаю Events».

☰ V2-C2-S02-C02 · Получить Pending без toleration и прочитать Events

```
kubectl --context "$CTX" apply -f "$LAB/postgres-pending.yaml"
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o
wide
kubectl --context "$CTX" -n traffic-lab describe pod -l app=devops-may-cry-
postgres | sed -n '/Events:/, $p'
```

✓ После команды

Ожидая: PostgreSQL остаётся `Pending`; Events одновременно показывают требование nodeAffinity и отсутствие toleration для taint database-ноды.

ГОВОРЮ

«В исправленном файле появляется toleration с тем же key, value и effect. Теперь два фильтра согласованы: affinity выбирает класс ноды, toleration разрешает вход.

ConfigMap передаёт SQL-инициализацию отдельно от образа».

ПОКАЗАТЬ

- `postgres-scheduled.yaml`: tolerations и nodeAffinity рядом.
- probes, requests/limits, non-root security context.
- `emptyDir` проговорить как ограничение этой лаборатории.

V2-C2-S02-C03 · Добавить toleration и запустить реальный PostgreSQL на node3

```
kubectl --context "$CTX" -n traffic-lab create configmap devops-may-cry-db-init \
  --from-file=001_init.sql="$LAB/001_init.sql" \
  --dry-run=client -o yaml | kubectl --context "$CTX" apply -f -
kubectl --context "$CTX" apply -f "$LAB/postgres-scheduled.yaml"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry-
postgres --timeout=120s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o
wide
```

После команды

Ожидаю: Pod `Running` именно на node3. Это настоящий PostgreSQL, но данные живут в `emptyDir`: для этой scheduler-лабы они намеренно одноразовые.

В ПРОДЕ

Боевой PostgreSQL обычно живёт в managed-сервисе или под оператором со StatefulSet, CSI/PVC, backup и проверенным restore. Здесь нужен живой процесс базы для Scheduler, поэтому storage одноразовый и это подписано прямо в манифесте.

ПЕРЕХОД

«Pod запущен на node3. Теперь удалю label и проверю, что означает слово Ignored в длинном имени правила».

Время: 26:00–34:00

ЭКРАН: Pod на node3 → удаление label → тот же Pod → пересоздание.

ГОВОРЮ

«Сначала фиксирую имя Pod и ноду. Удаляю label. Запущенный процесс остаётся на месте: Scheduler не пересматривает его каждую секунду, а kubelet не выселяет его из-за nodeAffinity».

V2-C2-S03-C01 · Показать смысл IgnoredDuringExecution

```
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o wide
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier-
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry-postgres -o wide
```

После команды

Ожидаю: уже запущенный PostgreSQL остаётся на node3, хотя label больше не соответствует required nodeAffinity.

ГОВОРЮ

«Удаляю Pod. Deployment создаёт новый, и вот для нового планирования label уже обязателен. Получаем Pending. Возвращаю label — условие снова выполнимо, Pod запускается».

📖 V2-C2-S03-C02 · Пересоздать Pod и вернуть label после доказанного Pending

```
OLD_POD="$(kubectl --context "$CTX" -n traffic-lab get pod \
  -l app=devops-may-cry-postgres -o jsonpath='{.items[0].metadata.name}')"
OLD_UID="$(kubectl --context "$CTX" -n traffic-lab get pod "$OLD_POD" \
  -o jsonpath='{.metadata.uid}')"
kubectl --context "$CTX" -n traffic-lab delete pod "$OLD_POD" --wait=true

NEW_POD=""
for _ in $(seq 1 60); do
  NEW_POD="$(kubectl --context "$CTX" -n traffic-lab get pod \
    -l app=devops-may-cry-postgres \
    -o jsonpath='{range .items[*]}{.metadata.uid}{ " "}{.metadata.name}{ " "}{.status.phase}{ "\n"}{end}' \
    | awk -v old="$OLD_UID" '$1 != old && $3 == "Pending" {print $2; exit}')"
  [ -n "$NEW_POD" ] && break
  sleep 1
done
test -n "$NEW_POD"
kubectl --context "$CTX" -n traffic-lab get pod "$NEW_POD" -o wide
kubectl --context "$CTX" -n traffic-lab describe pod "$NEW_POD" | sed -n
'/Events:/, $p'
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier=database
--overwrite
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry-
postgres --timeout=120s
kubectl --context "$CTX" -n traffic-lab get pod "$NEW_POD" -o wide
```

✅ После команды

Ожидаю: цикл находит именно новый UID в фазе `Pending`; Events показывают невозможную nodeAffinity. После возврата label тот же новый Pod становится `Running`.

ЧТО ЭТО ЗНАЧИТ

Affinity — входное условие Scheduler. Это не policy engine, который непрерывно двигает уже запущенные Pod'ы. Для постоянного контроля конфигурации нужны отдельные проверки и автоматика.

ПЕРЕХОД

«С базой закончили. Возвращаю node3 в общий worker-pool и перехожу к доступности stateless-приложения».

c2.4 Pod anti-affinity: полезное правило может заблокировать scale

Время: 34:00–48:00

ЭКРАН: base Deployment → required anti-affinity patch → Pod'ы на двух workers.

ГОВОРЮ

«Удаляю одноразовую базу, secret и configmap, снимаю taint и label. Это обязательный cleanup: в следующей части обе worker-ноды снова нужны приложению.

В base у DEVOPS MAY CRY мягкая anti-affinity и мягкий topology spread. Две реплики стараются разойтись, но правило не блокирует rollout».

V2-C2-S04-C01 · Вернуть workers в общий пул и разложить DEVOPS MAY CRY мягко

```
kubectl --context "$CTX" -n traffic-lab delete deploy/devops-may-cry-postgres
svc/devops-may-cry-postgres --ignore-not-found
kubectl --context "$CTX" -n traffic-lab delete secret/devops-may-cry-db
configmap/devops-may-cry-db-init --ignore-not-found
kubectl --context "$CTX" taint node node3 dedicated=database:NoSchedule-
2>/dev/null || true
kubectl --context "$CTX" label node node3 workload.boostmentor.dev/tier-
2>/dev/null || true
kubectl --context "$CTX" apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o wide
```

После команды

Ожидая: database-демо удалено, taint/label сняты, две реплики приложения Running и стараются разойтись по node2/node3.

ПОКАЗАТЬ

- `kubernetes/devops-may-cry/base/deployment.yaml`.
- `podAntiAffinity.preferred...`.
- `topologySpreadConstraints`, `topologyKey: kubernetes.io/hostname`.

ГОВОРЮ

«Если добавить required anti-affinity поверх живого RollingUpdate с `maxUnavailable: 0`, surge Pod сам может заблокировать rollout раньше нашей проверки. Поэтому для этой лаборатории делаю воспроизводимый reset: временно scale в ноль, жду удаления старых Pod'ов, применяю required-правило и сразу запускаю три новые реплики. Это лабораторный приём, а не рекомендация останавливать production. Workers только два. Первые две реплики размещаются, третьей физически негде выполнить правило».

📖 V2-C2-S04-C02 · Сделать anti-affinity жёсткой и увидеть цену третьей реплики

```
PATCH="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/17_2.4_pod_anti_affinity/required-anti-affinity-patch.yaml"
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=0
POD_COUNT=1
for _attempt in $(seq 1 30); do
    POD_COUNT="$(kubectl --context "$CTX" -n traffic-lab get pod \
        -l app=devops-may-cry --no-headers 2>/dev/null | wc -l | tr -d ' ')"
    test "$POD_COUNT" -eq 0 && break
    sleep 2
done
test "$POD_COUNT" -eq 0
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type merge
--patch-file "$PATCH"
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=3
RUNNING=0
PENDING=0
for _attempt in $(seq 1 30); do
    RUNNING="$(kubectl --context "$CTX" -n traffic-lab get pod \
        -l app=devops-may-cry --field-selector=status.phase=Running \
        --no-headers 2>/dev/null | wc -l | tr -d ' ')"
    PENDING="$(kubectl --context "$CTX" -n traffic-lab get pod \
        -l app=devops-may-cry --field-selector=status.phase=Pending \
        --no-headers 2>/dev/null | wc -l | tr -d ' ')"
    printf 'running=%s pending=%s\n' "$RUNNING" "$PENDING"
    test "$RUNNING" -eq 2 && test "$PENDING" -eq 1 && break
    sleep 2
done
test "$RUNNING" -eq 2
test "$PENDING" -eq 1
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o wide
```

```
kubectl --context "$CTX" -n traffic-lab describe pod -l app=devops-may-cry | sed -n '/Events:$/, $p'
```

✅ После команды

Ожидаю: после контролируемого lab-reset на двух workers работают две новые реплики, третья `Pending`; Events объясняют конфликт `required podAntiAffinity`.

🗣 ГОВОРЮ

«Это нормальный компромисс: `required` защищает от совместного падения, но может остановить масштабирование и rollout. Для небольшого кластера часто разумнее `preferred`, плюс PDB и запас capacity. Удаляю жёсткое поле и возвращаю две реплики».

📄 V2-C2-S04-C03 · Удалить жёсткое правило и вернуть базовые две реплики

```
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type=json \
-
p='[{"op":"remove","path":"/spec/template/spec/affinity/podAntiAffinity/requiredDuringSchedulingIgnoredDuringExecution"}]'
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=2
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
```

✅ После команды

Ожидаю: Deployment снова имеет две Ready-реплики; `required anti-affinity` удалена.

ПЕРЕХОД

«Anti-affinity отвечает на вопрос “не клади рядом с такими Pod'ами”. Topology spread формулирует цель иначе: держи измеримый перекоc не больше заданного».

C2.5 Topology spread: maxSkew считаем по фактическим нодам

Время: 48:00–58:00

ЭКРАН: `hard-spread-patch.yaml` → таблица Pod/NODE.

ГОВОРЮ

«Патч задаёт `topologyKey: kubernetes.io/hostname`, `maxSkew: 1` и `DoNotSchedule`. На четырёх репликах ожидаю два Pod'а на node2 и два на node3. На пяти — три и два. Не важны конкретные имена Pod'ов; важна разница между доменами».

V2-C2-S05-C01 · Измерить maxSkew на четырёх и пяти репликах

```
PATCH="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/18_2.5_topology_spread/hard-spread-patch.yaml"
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type merge --patch-file "$PATCH"
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=4
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry \
  -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName --sort-by=.spec.nodeName
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=5
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry \
  -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName --sort-by=.spec.nodeName
```

✅ После команды

Ожидаю: четыре Pod'a распределены `2+2`, пять — `3+2`; разница по hostname не больше `maxSkew: 1`.

🧠 ЧТО ЭТО ЗНАЧИТ

`maxSkew: 1` не означает "всегда поровну". Для нечётного числа реплик разница в одну допустима. При недоступной зоне `DoNotSchedule` может оставить Pod Pending; `ScheduleAnyway` разрешает работу ценой временного перекося.

☰ V2-C2-S05-C02 · Вернуть мягкий spread и две реплики

```
kubectl --context "$CTX" -n traffic-lab patch deploy devops-may-cry --type=json \
-
p='[{"op":"replace","path":"/spec/template/spec/topologySpreadConstraints/0/whenUn
satisfiable","value":"ScheduleAnyway"}]'
```

```
kubectl --context "$CTX" -n traffic-lab scale deploy devops-may-cry --replicas=2
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
```

✅ После команды

Ожидаю: приложение снова Ready с двумя репликами; spread стал предпочтением, а не причиной Pending.

ПЕРЕХОД

«До сих пор ограничения жили в PodСpec. Теперь поставим рамки на весь namespace команды».

c2.6 LimitRange и ResourceQuota работают в паре

Время: 58:00–68:00

ЭКРАН: `limitrange_resourcequota.yaml` → Pod resources → quota Used/Hard.

ГОВОРЮ

«LimitRange задаёт минимум, максимум и defaults для одного контейнера.

ResourceQuota считает суммарное потребление namespace. Это разные задачи.

Создаю отдельный `team-dev`, чтобы учебная квота не сломала `traffic-lab`.

Запускаю nginx без resources. Admission подставляет defaults из LimitRange до того, как quota посчитает запрос».

V2-C2-S06-C01 · Показать, как LimitRange подставляет ресурсы

```
LAB="$REPO_ROOT/02_ЧАСТЬ_2_SCHEDULER/19_2.6_limitrange_quota"
kubectl --context "$CTX" create namespace team-dev
kubectl --context "$CTX" -n team-dev apply -f
"$LAB/limitrange_resourcequota.yaml"
kubectl --context "$CTX" -n team-dev run naked --image=nginx:1.27
kubectl --context "$CTX" -n team-dev get pod naked -o
jsonpath='{.spec.containers[0].resources}'; echo
kubectl --context "$CTX" -n team-dev describe quota
```

После команды

Ожидая: у Pod без явных resources появились default requests/limits; ResourceQuota уже считает их в `Used`.

ПОКАЗАТЬ

- `defaultRequest` и `default`.
- суммарные `requests.cpu`, `limits.cpu`, число Pod/Service/PVC.

- лимит на `services.loadbalancers`: в облаке такой Service может стоить денег.

ГОВОРЮ

«Второй Pod просит 64 CPU. API отклоняет объект до запуска контейнера. Поэтому искать его application logs бессмысленно: процесса не было. После доказательства удаляю весь namespace».

V2-C2-S06-C02 · Получить отказ API на запрос выше квоты и удалить namespace

```
kubectl --context "$CTX" -n team-dev run fat --image=nginx:1.27 \
  --overrides='{"spec":{"containers":
[{"name":"fat","image":"nginx:1.27","resources":{"requests":{"cpu":"64"}}}]}}' ||
true
kubectl --context "$CTX" -n team-dev get events --sort-by=.lastTimestamp | tail
-12
kubectl --context "$CTX" delete namespace team-dev
```

После команды

Ожидаю: API отклоняет Pod как превышающий quota/LimitRange; namespace удалён вместе с учебными объектами.

ПЕРЕХОД

«Финальная привычка этой части: Pending — это статус, а не диагноз».

c2.7 Pending triage: три симптома, три причины

Время: 68:00–79:00

ЭКРАН: три Pod'a → Events каждого → cleanup.

ГОВОРЮ

«Создам три Pending одновременно. Первый требует несуществующий label `disktype=nvme`. Второй просит 64 CPU и 200 GiB памяти. Для третьего временно taint'ю оба workers, а toleration ему не даю.

`kubectl get pods` покажет один и тот же статус. `describe` в Events разделит причины. Это быстрее, чем наугад править image, CNI или приложение».

V2-C2-S07-C01 · Создать три разных Pending и получить три разных диагноза

```
kubectl --context "$CTX" create namespace pending-lab --dry-run=client -o yaml |
kubectl --context "$CTX" apply -f -
kubectl --context "$CTX" -n pending-lab run stuck-selector --image=nginx:1.27 \
  --overrides='{ "spec": { "nodeSelector": { "disktype": "nvme" } } }'
kubectl --context "$CTX" -n pending-lab run stuck-resources --image=nginx:1.27 \
  --overrides='{ "spec": { "containers": [ { "name": "stuck-
resources", "image": "nginx:1.27", "resources": { "requests":
{ "cpu": "64", "memory": "200Gi" } } } ] } }'
kubectl --context "$CTX" taint node node2 maintenance=true:NoSchedule
kubectl --context "$CTX" taint node node3 maintenance=true:NoSchedule
kubectl --context "$CTX" -n pending-lab run stuck-taint --image=nginx:1.27
kubectl --context "$CTX" -n pending-lab get pods
for pod in stuck-selector stuck-resources stuck-taint; do
  echo "=== $pod"
  kubectl --context "$CTX" -n pending-lab describe pod "$pod" | sed -n
  '/Events: /,$p'
done
```

✅ После команды

Ожидая: все три Pod'a `Pending`, но Events различаются: node selector, недостаток CPU/RAM и untolerated taint.

🧠 ЧТО ЭТО ЗНАЧИТ

Порядок triage короткий: Scheduler Events → node labels/taints → allocatable и requests → только потом более редкие причины. Если Pod ещё не назначен на ноду, пустые application logs ожидаемы.

🗣 ГОВОРЮ

«Перед переходом снимаю оба maintenance-taint и удаляю namespace. Затем проверяю, что DEVOPS MAY CRY снова Ready. Без этого следующая часть началась бы на испорченном стенде».

📋 V2-C2-S07-C02 · Обязательно вернуть ноды и удалить triage-namespace

```
kubectl --context "$CTX" taint node node2 maintenance=true:NoSchedule-
kubectl --context "$CTX" taint node node3 maintenance=true:NoSchedule-
kubectl --context "$CTX" delete namespace pending-lab
kubectl --context "$CTX" -n traffic-lab get deploy,po -l app=devops-may-cry -o
wide
```

✅ После команды

Ожидая: временные taints сняты, pending-lab удалён, DEVOPS MAY CRY остаётся Ready для части 3.

ПЕРЕХОД

«Scheduler выбрал ноду по requests и правилам размещения. В части 3 посмотрим, что после запуска делают limits, cgroups, OOM, throttling, HPA, VPA и Cluster Autoscaler».

Время: 00:00–12:00

ЭКРАН: доска, блок ЧЗ → VS Code, `03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server`.

ГОВОРЮ

«В предыдущей части мы разобрали, как scheduler выбирает ноду. Теперь посмотрим, что происходит после назначения: какие ограничения получает процесс, почему один контейнер замедляется, а другой завершается, и кто добавляет Pod'ы или ноды.

DEVOPS MAY CRY остаётся тем же приложением. Мы не меняем бинарник под каждый вывод. Меняем policy вокруг него: requests, limits, QoS, HPA и VPA. Так видно, что поведение создаёт Kubernetes-конфигурация, а не скрытая разница между тестовыми приложениями.

Масштабирование начинается не с HPA, а с измерения. В обычном кластере `kubectl top` не читает данные прямо из kubelet. Команда идёт в aggregated API `metrics.k8s.io`, который обслуживает Metrics Server. Он хранит короткий актуальный срез CPU и memory. Это не замена Prometheus: истории, алертов и произвольных метрик здесь нет.

Сначала фиксирую корень клона, два kube-context и pinned Kubespray. До изменения проверяю, существует ли Metrics API. Ошибка `Metrics API not available` в этой точке — ожидаемое состояние, а не поломка стенда».

ПОКАЗАТЬ ДО КОМАНДЫ

- `03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server/addons-metrics-server.yml`.
- `metrics_server_enabled: true`.

- Учебный параметр `metrics_server_kubelet_insecure_tls: true`.
- `kubespray/VERSION` — upstream закреплён, не скачивается из случайной ветки.

📖 V2-C3-S01-C01 · Зафиксировать контекст и состояние Metrics API до установки

```

REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT" || exit
set -a
source recording/.recording.env
set +a
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
KUBESPRAY_UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
KUBESPRAY_INVENTORY="$KUBESPRAY_ROOT/inventory/video2/inventory.ini"
ANSIBLE_PLAYBOOK="$KUBESPRAY_ROOT/.venv/bin/ansible-playbook"
SSH_KEY="${VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
test "$(git -C "$KUBESPRAY_UPSTREAM" describe --tags --exact-match)" = "v2.31.0"
kubectl --context "$SELF_CONTEXT" get nodes
kubectl --context "$SELF_CONTEXT" get apiservice v1beta1.metrics.k8s.io || true
kubectl --context "$SELF_CONTEXT" top nodes || true
sed -n '1,80p' 03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server/addons-metrics-server.yml

```

✅ После команды

Ожидаю: три Ready-ноды self-managed кластера; до установки Metrics API либо отсутствует, либо `kubectl top` сообщает, что API недоступен. Pinned upstream — ровно `v2.31.0`.

🗣️ ГОВОРЮ ПЕРЕД УСТАНОВКОЙ

«Копирую два параметра в `group_vars` нашего `inventory` и запускаю официальный `cluster.yml` только с тегом `metrics_server`. Ansible-вызов и выбранная роль остаются видны в кадре.

После `PLAY RECAP` проверяю три разных слоя. Deployment должен раскатиться. Aggregated APIService должен стать Available. И только затем сырой API и `kubectl top` должны вернуть samples».

☰ V2-C3-S01-C02 · Установить Metrics Server ролью Kubespray и доказать API

```
install -m 0644 \  
    "$REPO_ROOT/03_ЧАСТЬ_3_SCALING/21_3.0_metrics_server/addons-metrics-server.yml" \  
 \  
    "$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/addons.yml" \  
pushd "$KUBESPRAY_UPSTREAM" >/dev/null \  
"$ANSIBLE_PLAYBOOK" -i "$KUBESPRAY_INVENTORY" \  
    cluster.yml \  
    --become --private-key "$SSH_KEY" --tags metrics_server \  
popd >/dev/null \  
kubectl --context "$SELF_CONTEXT" -n kube-system \  
    rollout status deploy/metrics-server --timeout=180s \  
kubectl --context "$SELF_CONTEXT" wait --for=condition=Available \  
    apiservice/v1beta1.metrics.k8s.io --timeout=180s \  
kubectl --context "$SELF_CONTEXT" get apiservice v1beta1.metrics.k8s.io \  
kubectl --context "$SELF_CONTEXT" get --raw /apis/metrics.k8s.io/v1beta1/nodes \  
    | jq -r '.items[] | [.metadata.name,.usage.cpu,.usage.memory] | @tsv' \  
kubectl --context "$SELF_CONTEXT" top nodes \  
kubectl --context "$SELF_CONTEXT" top pods -A --sort-by=memory | head -15
```

✅ После команды

Ожидая: `PLAY RECAP` без failed, Deployment доступен, APIService имеет `Available=True`, сырой Metrics API и `kubectl top` возвращают samples.

ФИЗИЧЕСКИЙ СМЫСЛ

Metrics Server опрашивает kubelet summary API, а API Server публикует его через aggregation layer. HPA позже читает тот же resource metrics API. Сам `kubectl top` ничего не измеряет — он форматирует готовый ответ.

PRODUCTION NUANCE

В учебном стенде включён TLS-компромисс для self-signed kubelet serving certificates. В рабочей среде такой флаг не копируют автоматически: выпускают проверяемые serving-сертификаты, настраивают trust chain и проверяют APIService. Зелёный `kubectl top` также не подтверждает, что requests/limits выбраны правильно.

ПЕРЕХОД

«Метрики появились. Теперь откроем один Pod одновременно глазами scheduler и глазами Linux kernel».

с3.1 Requests для scheduler, limits для cgroup

Время: 12:00–25:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/22_3.1_requests_limits_cgroup/requests_limits.yaml` → терминал → SSH на worker.

ГОВОРЮ

«В `requests_limits.yaml` три Deployment используют один immutable image DEVOPS MAY CRY. Сейчас беру вариант `Burstable`: request `100m CPU` и `64Mi memory`, limit `500m` и `256Mi`.

Request отвечает на вопрос scheduler: на какой ноде зарезервировать место. Limit отвечает на другой вопрос: сколько процессу разрешит runtime. Scheduler не ждёт фактическую нагрузку и не читает cgroup. Он складывает requests из Pod spec.

Сначала применяю манифест, нахожу конкретный Pod и ноду. Затем показываю его QoS и секцию `Allocated resources` этой ноды».

ПОКАЗАТЬ

- Один и тот же digest image у всех трёх Deployment.
- У `devops-may-cry-burstable`: requests меньше limits.
- `automountServiceAccountToken: false`, non-root, dropped capabilities, read-only root filesystem.
- Не листать весь YAML: остановиться на `resources` и `securityContext`.

V2-C3-S02-C01 · Применить requests/limits и увидеть решение планировщика

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/22_3.1_requests_limits_cgroup"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" create namespace traffic-lab \
  --dry-run=client -o yaml | kubectl --context "$SELF_CONTEXT" apply -f -
kubectl --context "$SELF_CONTEXT" apply -f requests_limits.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  rollout status deploy/devops-may-cry-burstable --timeout=180s
POD="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry-burstable -o jsonpath='{.items[0].metadata.name}')"
NODE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$POD" \
  -o jsonpath='{.spec.nodeName}')"
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$POD" \
  -o custom-
columns='NAME:.metadata.name,NODE:.spec.nodeName,QOS:.status.qosClass,REQUEST_CPU:
.spec.containers[0].resources.requests.cpu,LIMIT_CPU:.spec.containers[0].resources.limits.cpu,REQUEST_MEM:.spec.containers[0].resources.requests.memory,LIMIT_MEM:
.spec.containers[0].resources.limits.memory'
kubectl --context "$SELF_CONTEXT" describe node "$NODE" \
  | sed -n '/Allocated resources:/,/Events:/p'
```

После команды

Ожидая: Pod `Burstable`, request `100m/64Mi`, limit `500m/256Mi`; в `Allocated resources` ноды requests и limits считаются отдельно.

ГОВОРЮ ПЕРЕД SSH

«Теперь проверю, во что kubelet и container runtime превратили limit. В контейнере нет shell и `cat`: образ distroless. Это не недостаток приложения, а уменьшение поверхности атаки.

Поэтому беру container ID из Pod status, а `nodeName` автоматически сопоставляю с `NODE1_SSH_HOST`, `NODE2_SSH_HOST` или `NODE3_SSH_HOST` из локального env. Адрес руками не угадываю. Через SSH и `crictl inspect` получаю PID и читаю cgroup из mount namespace процесса. Public IP нужен только для SSH. Сам Pod работает в private cluster network».

📄 V2-C3-S02-C02 · Прочитать реальные cgroup-ограничения через runtime

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab exec "$POD" \
  -- cat /sys/fs/cgroup/memory.max || true
CONTAINER_ID="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$POD" \
  -o jsonpath='{.status.containerStatuses[0].containerID}' | sed
's#^[^:]*://##')"
```

```
printf 'pod=%s node=%s container=%s\n' "$POD" "$NODE" "$CONTAINER_ID"
case "$NODE" in
  node1) NODE_SSH_HOST="{NODE1_SSH_HOST:?fill NODE1_SSH_HOST}" ;;
  node2) NODE_SSH_HOST="{NODE2_SSH_HOST:?fill NODE2_SSH_HOST}" ;;
  node3) NODE_SSH_HOST="{NODE3_SSH_HOST:?fill NODE3_SSH_HOST}" ;;
  *) printf 'unexpected node: %s\n' "$NODE" >&2; exit 1 ;;
esac
SSH_USER="{NODE02_SSH_USER:-root}"
REMOTE_PID="$(ssh -i "$SSH_KEY" "$SSH_USER@$NODE_SSH_HOST" \
  "sudo crictl inspect '$CONTAINER_ID'" \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["info"]["pid"])')"
```

```
ssh -i "$SSH_KEY" "$SSH_USER@$NODE_SSH_HOST" \
  "sudo nsenter -t '$REMOTE_PID' -m cat /sys/fs/cgroup/memory.max; sudo nsenter -
t '$REMOTE_PID' -m cat /sys/fs/cgroup/cpu.max"
```

✅ После команды

Ожидая: `kubectl exec` не находит `cat` в distroless-образе; через runtime видны конечный `memory.max` для 256Mi и CPU quota, соответствующая limit `500m`.

ФИЗИЧЕСКИЙ СМЫСЛ

Для cgroup v2 `memory.max` — предел памяти в байтах. `cpu.max` содержит quota и period: при `500m` контейнер получает примерно половину одного CPU во времени. Kubernetes Quantity из YAML превратился в параметры Linux kernel.

ЛОВУШКА

Не устанавливать shell в production image ради диагностики. Для редких случаев есть ephemeral debug container, runtime-инструменты на ноде и отдельные debug-образы. Доступ к ноде должен быть ограничен и аудироваться.

ПЕРЕХОД

«Мы записали память с суффиксом `Mi`. Сейчас покажу, почему это не косметика».

Время: 25:00–32:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/23_3.2_memory_units/oom_cpu_demo.yaml`, затем терминал.

ГОВОРЮ

«Kubernetes использует формат Quantity. Для CPU `1` означает одно ядро, `1000m` — то же самое, `100m` — десятую часть. Для памяти bare integer — байты. Поэтому `memory: 128` означает не 128 мегабайт, а 128 байт.

`M` — десятичные мегабайты, `Mi` — двоичные mebibytes. Обычно для памяти в манифестах пишут `Mi` и `Gi`, чтобы не гадать о единицах.

В `oom_cpu_demo.yaml` нормальный limit записан как `96Mi`. Для контраста создаю один временный Pod того же DEVOPS MAY CRY image с limit `128`. Полный security context передаю в объекте, поэтому Pod проходит restricted policy namespace. Жду не абстрактный CrashLoop, а конкретный `OOMKilled` и exit code».

ПОКАЗАТЬ

- В `oom_cpu_demo.yaml`: `requests.memory: 64Mi`, `limits.memory: 96Mi`.
- Команду с ошибочным `"memory": "128"`.
- После запуска — jsonpath с исходным limit, reason и exit code.

■ V2-C3-S03-C01 · Доказать, что memory 128 означает 128 байт

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete pod bad-memory-unit \
  --ignore-not-found --wait=true
kubectl --context "$SELF_CONTEXT" -n traffic-lab run bad-memory-unit \
  --image='ghcr.io/boost-mentor/devops-may-
cry@sha256:41439418d570649055117277a53b17b91fb40dad33dc3fa6b8a151fe68f86ca7' \
  --restart=Always \
  --overrides='{"spec":{"automountServiceAccountToken":false,"securityContext":
{"runAsNonRoot":true,"runAsUser":65532,"runAsGroup":65532,"seccompProfile":
{"type":"RuntimeDefault"}},"containers":[{"name":"bad-memory-
unit","image":"ghcr.io/boost-mentor/devops-may-
cry@sha256:41439418d570649055117277a53b17b91fb40dad33dc3fa6b8a151fe68f86ca7"},"res
ources":{"limits":{"memory":"128"}},"securityContext":
{"allowPrivilegeEscalation":false,"capabilities":{"drop":
["ALL"]},"readOnlyRootFilesystem":true,"runAsNonRoot":true,"runAsUser":65532,"run
AsGroup":65532}}]}'
for _attempt in $(seq 1 30); do
  REASON="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod bad-memory-
unit \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'
2>/dev/null || true)"
  test "$REASON" = "OOMKilled" && break
  sleep 2
done
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod bad-memory-unit \
  -o jsonpath='limit={.spec.containers[0].resources.limits.memory}{" reason="}
{.status.containerStatuses[0].lastState.terminated.reason}{" exit="}
{.status.containerStatuses[0].lastState.terminated.exitCode}{"\n"}'
test "$REASON" = "OOMKilled"
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete pod bad-memory-unit --
wait=true
```

✅ После команды

Ожидая: в объекте записан limit `128`, процесс завершается `OOMKilled` с exit code `137`; после доказательства Pod удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

API Server не исправляет смысл значения за автора. Он принимает валидную Quantity, runtime создаёт почти нулевой cgroup limit, а kernel завершает процесс, который физически не может уместиться.

⚠️ PRODUCTION NUANCE

Такие ошибки лучше ловить до кластера: schema validation, policy engine и review правил на минимальные/максимальные ресурсы. Одной проверки, что YAML парсится, недостаточно.

ПЕРЕХОД

«Даже с правильными единицами нельзя делить всю память ноды между приложениями. Часть ресурсов Kubernetes оставляет самой машине».

с3.3 Capacity и Allocatable — не одно число

Время: 32:00–39:00

ЭКРАН: терминал, объект `Node/node2`; локального манифеста в этой сцене нет.

ГОВОРЮ

«Capacity — то, что kubelet увидел на машине. Allocatable — то, что нода объявляет доступным Pod'ам после системных резервов и eviction threshold. Scheduler планирует по Allocatable.

В этой лаборатории нет YAML, который создаёт Capacity: это status объекта Node, его заполняет kubelet. Сравниваю оба поля на `node2`, затем смотрю уже назначенные requests и limits».

V2-C3-S04-C01 · Сравнить Capacity и Allocatable на worker-ноде

```
kubectl --context "$SELF_CONTEXT" describe node node2 \
  | sed -n '/Capacity:\/,/System Info:/p'
kubectl --context "$SELF_CONTEXT" get node node2 \
  -o jsonpath='capacity={.status.capacity}{"\n"}allocatable={.status.allocatable}{"\n"}'
kubectl --context "$SELF_CONTEXT" describe node node2 \
  | sed -n '/Allocated resources:\/,/Events:/p'
```

После команды

Ожидая: Node публикует оба набора значений; Allocatable отражает доступное Pod'ам после резервов и порогов kubelet, а `Allocated resources` показывает сумму requests/limits уже назначенных Pod'ов.

ФИЗИЧЕСКИЙ СМЫСЛ

Упрощённо: $\text{Allocatable} = \text{Capacity} - \text{system reserved} - \text{kube reserved} - \text{eviction reserve}$. Точная разница зависит от конфигурации kubelet и ОС. Если reservations не настроены, системные процессы всё равно потребляют ресурсы, но scheduler может этого не учесть достаточно консервативно.

ЛОВУШКА

Нельзя обещать приложениям 100% RAM ноды. Kernel, runtime, kubelet и системные DaemonSet тоже должны пережить пик. Для capacity planning смотрят Allocatable, DaemonSet overhead и реальное использование, а не только паспорт VM.

ПЕРЕХОД

«Requests должны помещаться в Allocatable. А вот limits Kubernetes разрешает продать несколько раз».

Время: 39:00–49:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/25_3.4_overcommit/requests_limits.yaml` → терминал.

ГОВОРЮ

«У Burstable-варианта request CPU `100m`, limit `500m`. Масштабирую его до двенадцати реплик на двух workers.

Scheduler проверяет сумму requests. Поэтому Pod'ы могут разместиться, даже если сумма limits на ноде превышает 100%. Это overcommit: мы рассчитываем, что не все контейнеры одновременно потребуют максимум.

После rollout показываю `Allocated resources` на node2 и node3. Затем удаляю только три Deployment этой лабы, чтобы следующий отказ не смешивался с фоновыми Pod'ами».

V2-C3-S05-C01 · Показать overcommit без искусственного отказа ноды

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/25_3.4_overcommit"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f requests_limits.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    scale deploy/devops-may-cry-burstable --replicas=12
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry-burstable --timeout=180s
for node in node2 node3; do
    printf '\n=== %s ===\n' "$node"
    kubectl --context "$SELF_CONTEXT" describe node "$node" \
        | sed -n '/Allocated resources:/,/Events:/p'
done
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pods \
    -l app=devops-may-cry-burstable -o wide
kubectl --context "$SELF_CONTEXT" delete -f requests_limits.yaml --ignore-not-found
```

После команды

Ожидая: все 12 Pod'ов планируются, потому что помещается сумма requests; сумма CPU limits на worker-нодах может быть выше 100%. В конце три лабораторных Deployment удалены.

ФИЗИЧЕСКИЙ СМЫСЛ

CPU — compressible resource: при конкуренции процессы получают меньше CPU и работают дольше. Memory — non-compressible: если физическая память закончилась, кто-то будет reclaim'иться, evict'иться или получит OOM. Поэтому одинаковый процент overcommit для CPU и memory имеет разный риск.

⚠ **PRODUCTION NUANCE**

Overcommit выбирают по профилю нагрузки, p95/p99, SLO и запасу ноды. Если все ночные batch-задачи стартуют одновременно, предположение «пики не совпадут» перестаёт работать. Scarcity policy должна учитывать такой сценарий.

ПЕРЕХОД

«Теперь специально превысим memory limit одного контейнера и отделим cgroup OOM от нехватки памяти на всей ноде».

Время: 49:00–61:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/26_3.5_oomkilled/oom_cpu_demo.yaml` → терминал → коротко SSH.

ГОВОРЮ

«В первом документе `oom_cpu_demo.yaml` DEVOPS MAY CRY имеет request `64Mi` и limit `96Mi`. Endpoint `/wound` включён только переменной `LAB_ENDPOINTS_ENABLED=true` в этой изолированной лабе. В обычном base он недоступен.

Приложение выделяет 160Mi и касается каждой страницы памяти. Это важно: просто создать большой slice недостаточно, пока kernel физически не закоммитил страницы.

Применяю файл, оставляю только OOM-вариант, фиксирую restartCount и вызываю `/wound` одноразовым контейнером с тем же image. Жду, пока restartCount действительно увеличится».

ПОКАЗАТЬ

- `LAB_ENDPOINTS_ENABLED: "true"`.
- `limits.memory: 96Mi`.
- Readiness probe и restricted security context.
- Второй Deployment в файле существует для следующей сцены; сейчас он удаляется.

📄 V2-C3-S06-C01 · Воспроизвести cgroup OOM на DEVOPS MAY CRY

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/26_3.5_oomkilled"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f oom_cpu_demo.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    delete deploy devops-may-cry-throttled --ignore-not-found
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry-oom --timeout=180s
OOM_POD="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
    -l app=devops-may-cry-oom -o jsonpath='{.items[0].metadata.name}')"
OOM_IP="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$OOM_POD" \
    -o jsonpath='{.status.podIP}')"
IMAGE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get deploy devops-may-
cry-oom \
    -o jsonpath='{.spec.template.spec.containers[0].image}')"
RESTARTS_BEFORE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod
"$OOM_POD" \
    -o jsonpath='{.status.containerStatuses[0].restartCount}')"
kubectl --context "$SELF_CONTEXT" -n traffic-lab run wound-trigger \
    --rm -i --restart=Never --image="$IMAGE" \
    --env="HEALTHCHECK_URL=http://$OOM_IP:8080/wound?mb=160" \
    --command -- /devops-may-cry -healthcheck || true
for _attempt in $(seq 1 30); do
    RESTARTS_AFTER="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod
"$OOM_POD" \
        -o jsonpath='{.status.containerStatuses[0].restartCount}')"
    test "$RESTARTS_AFTER" -gt "$RESTARTS_BEFORE" && break
    sleep 2
done
test "$RESTARTS_AFTER" -gt "$RESTARTS_BEFORE"
```

✅ После команды

Ожидая: запрос к лабораторному `/wound` превышает limit `96Mi`, соединение может оборваться, а restartCount основного Pod'а увеличивается.

🗣 ГОВОРЮ ПЕРЕД ДОКАЗАТЕЛЬСТВОМ

«Оборванный HTTP-запрос ещё ничего не доказывает. Читаю `Last State` контейнера: reason должен быть `OOMKilled`, exit code — `137`. Это `128 + 9`, где 9 — `SIGKILL`.

Дополнительно смотрю kernel log на той ноде, которую вернул Pod spec; её SSH-адрес команда берёт из локального env. Эта строка зависит от ОС и политики доступа, поэтому переносимый acceptance criterion — Pod status. После проверки удаляю OOM workload».

📄 V2-C3-S06-C02 · Доказать OOMKilled в Pod status и kernel log

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab describe pod "$OOM_POD" \
  | sed -n '/Last State:\/,/Ready:/p'
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$OOM_POD" \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}{"
exit="}{.status.containerStatuses[0].lastState.terminated.exitCode}{" restarts="}
{.status.containerStatuses[0].restartCount}{"\n"}'
NODE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$OOM_POD" \
  -o jsonpath='{.spec.nodeName}')"
case "$NODE" in
  node1) NODE_SSH_HOST="${NODE1_SSH_HOST:?fill NODE1_SSH_HOST}" ;;
  node2) NODE_SSH_HOST="${NODE2_SSH_HOST:?fill NODE2_SSH_HOST}" ;;
  node3) NODE_SSH_HOST="${NODE3_SSH_HOST:?fill NODE3_SSH_HOST}" ;;
  *) printf 'unexpected node: %s\n' "$NODE" >&2; exit 1 ;;
esac
SSH_USER="${VIDE02_SSH_USER:-root}"
ssh -i "$SSH_KEY" "$SSH_USER@$NODE_SSH_HOST" \
  'sudo dmesg -T | grep -iE "memory cgroup out of memory|killed process" | tail
-5' || true
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete deploy devops-may-cry-oom --ignore-not-found
```

✅ После команды

Ожидая: `Last State` содержит `Reason: OOMKilled` и exit code `137`; kernel log может добавить строку cgroup OOM, но главным переносимым доказательством остаётся Pod status. Workload удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Здесь нода могла иметь свободную память. Процесс завершил memory controller именно его cgroup, потому что контейнер превысил собственный limit. kubelet затем перезапустил контейнер внутри того же Pod по restart policy Deployment.

⚠ ЛОВУШКА

Exit `137` часто указывает на SIGKILL, но сам по себе не называет источник. Сверяем `lastState.terminated.reason`, события, метрики и kernel log. `kubectl logs` текущего контейнера без `--previous` может быть пустым или показывать уже новый процесс.

ПЕРЕХОД

«С памятью процесс завершился. На CPU limit результат другой: процесс останется жив, но будет ждать quota».

Время: 61:00–72:00

ЭКРАН: второй документ

`03_ЧАСТЬ_3_SCALING/27_3.6_cpu_throttling/oom_cpu_demo.yaml` → терминал.

ГОВОРЮ

«У throttled-варианта request `100m`, limit `200m`. Endpoint `/overload` запускает две CPU-bound goroutine на 120 секунд. Контейнер попытается взять больше quota, чем разрешено.

До нагрузки читаю cAdvisor counter `container_cpu_cfs_throttled_periods_total`. После запроса жду двадцать секунд и читаю тот же counter. Проверяю два условия: значение выросло, restartCount остался нулём».

ПОКАЗАТЬ

- `limits.cpu: 200m`.
- Тот же digest DEVOPS MAY CRY.
- Команду выборки одного Pod из cAdvisor metrics по container и pod label.

■ V2-C3-S07-C01 · Доказать CPU throttling счётчиком, а не перезапуском

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/27_3.6_cpu_throttling"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f oom_cpu_demo.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    delete deploy devops-may-cry-oom --ignore-not-found
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry-throttled --timeout=180s
CPU_POD="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
    -l app=devops-may-cry-throttled -o jsonpath='{.items[0].metadata.name}')"
CPU_IP="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$CPU_POD" \
    -o jsonpath='{.status.podIP}')"
IMAGE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get deploy devops-may-
cry-throttled \
    -o jsonpath='{.spec.template.spec.containers[0].image}')"
NODE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$CPU_POD" \
    -o jsonpath='{.spec.nodeName}')"
BEFORE="$(kubectl --context "$SELF_CONTEXT" get --raw
"/api/v1/nodes/$NODE/proxy/metrics/cadvisor" \
    | grep 'container_cpu_cfs_throttled_periods_total' \
    | grep 'container="devops-may-cry"' | grep "pod=\"$CPU_POD\"" \
    | awk '{print $2}' | head -1)"
test -n "$BEFORE"
kubectl --context "$SELF_CONTEXT" -n traffic-lab run overload-trigger \
    --rm -i --restart=Never --image="$IMAGE" \
    --env="HEALTHCHECK_URL=http://$CPU_IP:8080/overload?sec=120" \
    --command -- /devops-may-cry -healthcheck
sleep 20
kubectl --context "$SELF_CONTEXT" -n traffic-lab top pod "$CPU_POD"
AFTER="$(kubectl --context "$SELF_CONTEXT" get --raw
"/api/v1/nodes/$NODE/proxy/metrics/cadvisor" \
    | grep 'container_cpu_cfs_throttled_periods_total' \
    | grep 'container="devops-may-cry"' | grep "pod=\"$CPU_POD\"" \
```

```
| awk '{print $2}' | head -1)"
RESTARTS="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$CPU_POD" \
-o jsonpath='{.status.containerStatuses[0].restartCount}')"
printf 'throttled periods: before=%s after=%s; restarts=%s\n' \
"$BEFORE" "$AFTER" "$RESTARTS"
awk -v before="$BEFORE" -v after="$AFTER" 'BEGIN { exit !(after > before) }'
test "$RESTARTS" -eq 0
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
delete deploy devops-may-cry-throttled --ignore-not-found
```

✅ После команды

Ожидая: throttled-period counter увеличивается, Pod остаётся Running, restartCount равен нулю. CPU limit замедлил процесс, но не убил его.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Linux CFS выдаёт процессу quota на period. Когда quota исчерпана, runnable thread ждёт следующего периода. Контейнер остаётся Running, но latency растёт. Поэтому «Pod не перезапускается» не означает «CPU limit не влияет».

⚠️ PRODUCTION NUANCE

Искать throttling только через `kubectl top` неудобно: top показывает usage sample, а не время ожидания quota. Для расследования нужны throttling counters рядом с latency, saturation и requests. Иногда CPU limit снимают, но это решение принимают по профилю нагрузки, а не как универсальное правило.

ПЕРЕХОД

«Requests и limits также определяют QoS-класс. Проверим классы на одном image, а потом разберём, чего QoS не гарантирует».

Время: 72:00–82:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/28_3.7_qos_oom/requests_limits.yaml` → терминал.

ГОВОРЮ

«В файле три Deployment одного приложения.

`Guaranteed`: у каждого контейнера заданы CPU и memory, request равен limit.

`BestEffort`: requests и limits нет вообще. Всё остальное попадает в `Burstable`.

Применяю файл и проверяю каждый класс как acceptance criterion. Не делаю вывод по имени Deployment — читаю `.status.qosClass`, который присвоил kubelet».

V2-C3-S08-C01 · Проверить три QoS-класса на одном image

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/28_3.7_qos_oom"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -f requests_limits.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab wait --for=condition=Available \
  deploy/devops-may-cry-guaranteed \
  deploy/devops-may-cry-burstable \
  deploy/devops-may-cry-besteffort --timeout=180s
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pods \
  -l lab.boostmentor.dev/qos \
  -o custom-
columns='NAME:.metadata.name,QOS:.status.qosClass,NODE:.spec.nodeName'
for expected in \
  guaranteed:Guaranteed \
  burstable:Burstable \
  best-effort:BestEffort; do
  label="${expected%*:}"
  wanted="${expected##*:}"
  pod="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
    -l "lab.boostmentor.dev/qos=$label" \
    -o jsonpath='{.items[0].metadata.name}')"
  actual="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod "$pod" \
    -o jsonpath='{.status.qosClass}')"
  printf '%-12s expected=%-10s actual=%s\n' "$label" "$wanted" "$actual"
  test "$actual" = "$wanted"
done
kubectl --context "$SELF_CONTEXT" delete -f requests_limits.yaml --ignore-not-found
```

✅ После команды

Ожидая: один и тот же immutable image получает классы `Guaranteed`, `Burstable`, `BestEffort` только из-за разных resources; все три проверки проходят, затем workload удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

QoS влияет на параметры cgroup и на поведение kubelet при pressure. Но фраза «BestEffort всегда умирает первым» слишком грубая. При memory pressure kubelet учитывает, превышает ли Pod request, его Priority и насколько usage превышает request. При cgroup OOM внутри собственного limit картина снова другая.

⚠️ ЛОВУШКА

Guaranteed не означает «никогда не завершится» и не создаёт дополнительную память. Он фиксирует reservation и limit. Для критичного workload дополнительно нужны PriorityClass, topology, PDB, capacity и нормальные probes.

ПЕРЕХОД

«OOMKilled был решением memory controller для процесса. Eviction — решение kubelet для Pod. Получим второй результат отдельно».

с3.8 Evicted — это не OOMKilled

Время: 82:00–92:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/29_3.8_eviction/eviction-demo.yaml` → терминал.

ГОВОРЮ

«Я не буду забивать память или диск всей worker-ноды. Лаба ограничена одним Pod.

В `eviction-demo.yaml` контейнер пишет 64Mi в `emptyDir`. У volume и container ephemeral-storage limit — 16Mi. `emptyDir.sizeLimit` не создаёт отдельный маленький диск; kubelet периодически считает local ephemeral storage и, когда Pod превышает лимит, evict'ит его.

До запуска показываю, что у нод нет MemoryPressure, DiskPressure и PIDPressure.

Затем применяю один Pod и жду конкретный `.status.reason=Evicted`».

ПОКАЗАТЬ

- `restartPolicy: Never`.
- `requests.ephemeral-storage: 8Mi`, `limits: 16Mi`.
- `emptyDir.sizeLimit: 16Mi`.
- Restricted security context и pinned debug image.

📄 V2-C3-S09-C01 · Получить Evicted без давления на всю ноду

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/29_3.8_eviction"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" get nodes \
  -o custom-columns='NAME:.metadata.name,MEMORY:.status.conditions[?
(@.type=="MemoryPressure")].status,DISK:.status.conditions[?
(@.type=="DiskPressure")].status,PID:.status.conditions[?
(@.type=="PIDPressure")].status'
sed -n '1,220p' eviction-demo.yaml
kubectl --context "$SELF_CONTEXT" apply -f eviction-demo.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab wait \
  --for=jsonpath='{.status.reason}'=Evicted \
  pod/ephemeral-storage-hog --timeout=300s
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod ephemeral-storage-hog \
  -o jsonpath='phase={.status.phase}{" reason="}{.status.reason}{"\nmessage="}
{.status.message}{"\n"}'
test "$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod ephemeral-
storage-hog \
  -o jsonpath='{.status.reason}')" = "Evicted"
kubectl --context "$SELF_CONTEXT" -n traffic-lab get events \
  --field-selector involvedObject.name=ephemeral-storage-hog \
  --sort-by=.lastTimestamp
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete pod ephemeral-storage-hog --ignore-not-found
```

✅ После команды

Ожидаю: отдельный Pod получает phase `Failed`, reason `Evicted` из-за превышения своего ephemeral-storage limit; NodePressure не создаётся намеренно. Pod удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Eviction переводит Pod в phase `Failed` и reason `Evicted`. Это объект уровня kubelet/Pod, не lastState `OOMKilled` одного процесса. Поскольку здесь standalone Pod, controller не создаёт замену. У Deployment на его месте ReplicaSet создал бы новый Pod.

⚠️ PRODUCTION NUANCE

Причиной eviction может быть node pressure или Pod-level превышение local storage. Поэтому рядом с reason читают message, events и Node conditions. Массовое удаление всех Failed Pod'ов стирает доказательства и затрагивает чужие workload — здесь удаляем только свой Pod.

ПЕРЕХОД

«Теперь у нас есть метрики и корректные requests. Этого достаточно, чтобы HPA посчитал нужное число реплик».

с3.9 HPA: больше Pod'ов по CPU utilization

Время: 92:00–106:00

ЭКРАН: `03_ЧАСТЬ_3_SCALING/30_3.9_hpa/app` → `hpa_lab.yaml` → терминал.

ГОВОРЮ

«Base DEVOPS MAY CRY выглядит как обычный workload: Deployment, ClusterIP Service, PDB, probes, requests/limits и restricted security context. Endpoint нагрузки в base выключен. Overlay `recording` добавляет только `LAB_ENDPOINTS_ENABLED=true` для этой сцены.

В `hpa_lab.yaml` target — Deployment, минимум две реплики, максимум шесть, CPU utilization 50%. Utilization считается относительно CPU request контейнера. Без request HPA по этой метрике не сможет посчитать процент.

Сначала применяю overlay, отдельный debug client и HPA. Проверяю baseline: две Ready-реплики, request `100m`, HPA видит метрику».

ПОКАЗАТЬ

- `app/base/deployment.yaml`: resources и probes.
- `app/overlays/recording/enable-lab-endpoints.yaml`: одна env-переменная.
- `hpa_lab.yaml`: min/max, target 50%, scaleUp и scaleDown behavior.
- `debug-client.yaml`: отдельный ограниченный Pod, а не shell внутри production image.

📑 V2-C3-S10-C01 · Подготовить HPA с Metrics API и контролируемым endpoint

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/30_3.9_hpa"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -k app/overlays/recording
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$SELF_CONTEXT" apply -f debug-client.yaml
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  wait --for=condition=Ready pod/client --timeout=180s
kubectl --context "$SELF_CONTEXT" apply -f hpa_lab.yaml
CURRENT_UTILIZATION=""
for _attempt in $(seq 1 18); do
  CURRENT_UTILIZATION="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    get hpa devops-may-cry \
    -o jsonpath='{.status.currentMetrics[0].resource.current.averageUtilization}'
  \
    2>/dev/null || true)"
  test -n "$CURRENT_UTILIZATION" && break
  sleep 5
done
test -n "$CURRENT_UTILIZATION"
kubectl --context "$SELF_CONTEXT" -n traffic-lab get hpa devops-may-cry
kubectl --context "$SELF_CONTEXT" -n traffic-lab get deploy devops-may-cry \
  -o custom-
columns='NAME:.metadata.name,REPLICAS:.status.readyReplicas,REQUEST_CPU:.spec.template.spec.containers[0].resources.requests.cpu'
```

✅ После команды

Ожидая: Deployment имеет две Ready-реплики и CPU request `100m`; HPA видит текущую метрику, target `50%`, min `2`, max `6`.

ГОВОРЮ ПЕРЕД НАГРУЗКОЙ

«Из debug client отправляю восемь запросов к `/overload`. Endpoint быстро отвечает, а CPU-работа продолжается девяносто секунд внутри Pod'ов.

Упрощённая формула HPA: $\text{ceil}(\text{current replicas} \times \text{current utilization} / \text{target utilization})$. Контроллер применяет tolerance и behavior policy, поэтому число меняется не мгновенно и не обязано перескочить сразу на максимум.

Жду, пока desiredReplicas станет больше двух, показываю новые Pod'ы и затем возвращаю стенд к baseline».

V2-C3-S10-C02 · Создать CPU-нагрузку и доказать горизонтальное масштабирование

```
kubectl --context "$SELF_CONTEXT" -n traffic-lab exec client -- sh -c \
  'for i in $(seq 1 8); do curl -fsS "http://devops-may-cry/overload?sec=90"
  >/dev/null & done; wait'
for _attempt in $(seq 1 24); do
  kubectl --context "$SELF_CONTEXT" -n traffic-lab get hpa devops-may-cry
  DESIRED="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get hpa devops-may-
  cry \
    -o jsonpath='{.status.desiredReplicas}')"
  test "${DESIRED:-0}" -gt 2 && break
  sleep 5
done
test "$DESIRED" -gt 2
kubectl --context "$SELF_CONTEXT" -n traffic-lab get pods \
  -l app=devops-may-cry -o wide
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete hpa devops-may-cry
kubectl --context "$SELF_CONTEXT" -n traffic-lab scale \
  deploy/devops-may-cry --replicas=2
kubectl --context "$SELF_CONTEXT" -n traffic-lab delete pod client --wait=true
```

✅ После команды

Ожидая: CPU utilization превышает target, desiredReplicas становится больше двух, появляются дополнительные Pod'ы, но не больше шести. После доказательства HPA и клиент удалены, Deployment возвращён к двум репликам.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

HPA меняет поле replicas у target workload. Он создаёт Pod'ы через Deployment/ReplicaSet, но не создаёт VM. Если новым Pod'ам некуда сесть, они останутся Pending — это уже задача Cluster Autoscaler.

⚠️ ЛОВУШКА

Один короткий spike, отсутствующий request или запаздывающий Metrics API дают другой результат. Scale-down обычно стабилизируют дольше, чтобы реплики не дёргались туда-сюда. В рабочей среде target выбирают по latency, saturation и стоимости, а не потому что 50% выглядит аккуратно.

ПЕРЕХОД

«HPA меняет количество Pod'ов. VPA отвечает на другой вопрос: какой request разумно дать одному Pod'у».

Время: 106:00–120:00 плюс ожидание samples

ЭКРАН: `03_ЧАСТЬ_3_SCALING/31_3.10_vpa/vpa_lab.yaml` → pinned upstream manifests → терминал.

ГОВОРЮ

«VPA состоит из CRD и контроллеров. Для этой сцены нужны CRD, RBAC и recommender. Updater и admission-controller не ставлю: я не хочу, чтобы лаборатория сама пересоздала workload или переписала request.

Source закреплён одновременно тегом `vertical-pod-autoscaler-1.7.0` и точным commit. Перед apply показываю upstream `recommender-deployment.yaml`, затем применяю конкретные CRD/RBAC/recommender manifests. Каждый устанавливаемый объект остаётся виден».

ПОКАЗАТЬ

- `vpa_lab.yaml`: targetRef на DEVOPS MAY CRY.
- `updateMode: "Off"`.
- minAllowed/maxAllowed и controlledResources.
- В терминале — exact tag/commit и отсутствие updater/admission-controller.

📄 V2-C3-S11-C01 · Установить только VPA recommender из pinned source

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/31_3.10_vpa"
cd "$LAB_DIR" || exit
kubectl --context "$SELF_CONTEXT" apply -k base
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    delete hpa devops-may-cry --ignore-not-found
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry --timeout=180s
VPA_VERSION="1.7.0"
VPA_COMMIT="6a616ea0c5ea0cb6111240073a9273b3467c064e"
VPA_SOURCE="${XDG_CACHE_HOME:-$HOME/.cache}/video2-vpa/autoscaler"
if [[ ! -d "$VPA_SOURCE/.git" ]]; then
    mkdir -p "$(dirname "$VPA_SOURCE")"
    git clone --depth 1 --branch "vertical-pod-autoscaler-$VPA_VERSION" \
        https://github.com/kubernetes/autoscaler.git "$VPA_SOURCE"
fi
test "$(git -C "$VPA_SOURCE" rev-parse HEAD)" = "$VPA_COMMIT"
VPA_DEPLOY="$VPA_SOURCE/vertical-pod-autoscaler/deploy"
test -z "$(kubectl --context "$SELF_CONTEXT" -n kube-system get deploy \
    vpa-updater vpa-admission-controller --ignore-not-found -o name)"
sed -n '1,80p' "$VPA_DEPLOY/recommender-deployment.yaml"
kubectl --context "$SELF_CONTEXT" apply -f "$VPA_DEPLOY/vpa-v1-crd-gen.yaml"
kubectl --context "$SELF_CONTEXT" apply -f "$VPA_DEPLOY/vpa-rbac.yaml"
kubectl --context "$SELF_CONTEXT" apply -f "$VPA_DEPLOY/recommender-
deployment.yaml"
kubectl --context "$SELF_CONTEXT" -n kube-system \
    rollout status deploy/vpa-recommender --timeout=180s
kubectl --context "$SELF_CONTEXT" get crd \
    verticalpodautoscalers.autoscaling.k8s.io
```

✅ После команды

Ожидаю: source имеет exact commit, установлены CRD/RBAC/recommender; updater и admission-controller отсутствуют, recommender доступен.

🗣 ГОВОРЮ ПЕРЕД ОЖИДАНИЕМ

«Перед созданием VPA сохраняю UID текущих Pod'ов. Recommender нужна история metrics, поэтому опрашиваю не просто существование объекта, а непустую target recommendation.

Когда target CPU/memory появился, сравниваю UID. В `off` режиме они должны совпасть: VPA посчитал рекомендацию, но ничего не применил».

📄 V2-C3-S11-C02 · Получить рекомендацию VPA без пересоздания Pod'ов

```
PODS_BEFORE="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry \
  -o jsonpath='{range .items[*]}{.metadata.uid}{"\n"}{end}' | sort | tr '\n' '
  ')"
kubectl --context "$SELF_CONTEXT" apply -f vpa_lab.yaml
VPA_TARGET=""
for attempt in $(seq 1 36); do
  VPA_TARGET="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get vpa devops-
may-cry \
  -o jsonpath='{.status.recommendation.containerRecommendations[0].target.cpu}
{" / "}{.status.recommendation.containerRecommendations[0].target.memory}' \
  2>/dev/null || true)"
  test "$VPA_TARGET" != " / " && test -n "$VPA_TARGET" && break
  printf 'waiting for VPA recommendation (%s/36)\r' "$attempt"
  sleep 10
done
test "$VPA_TARGET" != " / "
printf 'VPA target cpu / memory: %s\n' "$VPA_TARGET"
kubectl --context "$SELF_CONTEXT" -n traffic-lab describe vpa devops-may-cry \
  | sed -n '/Recommendation:/, $p'
PODS_AFTER="$(kubectl --context "$SELF_CONTEXT" -n traffic-lab get pod \
  -l app=devops-may-cry \
  -o jsonpath='{range .items[*]}{.metadata.uid}{"\n"}{end}' | sort | tr '\n' '
  ')"
printf 'pod UIDs before: %s\npod UIDs after:  %s\n' "$PODS_BEFORE" "$PODS_AFTER"
test "$PODS_BEFORE" = "$PODS_AFTER"
kubectl --context "$SELF_CONTEXT" -n traffic-lab \
  delete vpa devops-may-cry --ignore-not-found
```

✅ После команды

Ожидаю: VPA публикует target CPU/memory; UID двух Pod'ов до и после совпадают, потому что `updateMode: Off`. Объект VPA удалён.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

VPA анализирует usage samples и предлагает requests. Рекомендация — вход для инженерного решения, а не готовый production values-файл. Её сверяют с p95/p99, startup, OOM, throttling и сезонностью.

⚠️ ЛОВУШКА

HPA по CPU utilization делит usage на request. Если VPA одновременно меняет request, горизонтальная формула тоже меняется. Совместную политику проектируют явно: например, VPA рекомендует, а человек обновляет requests; либо HPA масштабирует по метрике, которая не конфликтует с изменяемым request.

ПЕРЕХОД

«Pod'ы мы масштабировать умеем. Остался случай, когда третий Pod корректный, но двум нодам физически некуда его поставить».

Время: 120:00–139:00 плюс создание VM

ЭКРАН: `infra/managed/terraform/kubernetes.tf` →

`03_ЧАСТЬ_3_SCALING/32_3.11_cluster_autoscaler/devops_may_cry.yaml` → Yandex Cloud UI и терминал.

ГОВОРЮ

«Для Cluster Autoscaler переключаюсь на managed-кластер. Self-managed CA тоже возможен, но ему нужен provider integration и права создавать ноды. В managed node group эту интеграцию обслуживает облако.

Продолжаю тот же Terraform state, которым managed-кластер создан в части 1. Второй одноимённый кластер из другого state не создаю. Перед plan обновляю IAM-токен в текущем терминале: с момента первой части могло пройти больше срока его жизни.

В `infra/managed/terraform/kubernetes.tf` один dynamic block выбирает fixed scale, другой — auto scale. Включаю диапазон от двух до пяти нод. Plan должен менять существующую node group, а не собирать новый кластер. После apply проверяю baseline — ровно две Ready-ноды».

ПОКАЗАТЬ

- `enable_autoscaling` в `main.tf`.
- `fixed_scale` и `auto_scale` в `kubernetes.tf`.
- `min=2`, `max=5`, `initial=node_count`.
- В Yandex Cloud UI — node group и autoscaling range после apply.

V2-C3-S12-C01 · Включить Cluster Autoscaler в существующей managed node group

```
MANAGED_TF="$REPO_ROOT/infra/managed/terraform"
cd "$MANAGED_TF" || exit
export YC_TOKEN="$(yc iam create-token)"
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN" && test -n "$YC_CLOUD_ID" && test -n "$YC_FOLDER_ID"
terraform init
terraform validate
terraform plan \
    -var='enable_autoscaling=true' \
    -var='autoscaling_min_nodes=2' \
    -var='autoscaling_max_nodes=5' \
    -out=/tmp/video2-cluster-autoscaler.tfplan \
    >/tmp/video2-cluster-autoscaler-plan.log
terraform show -json /tmp/video2-cluster-autoscaler.tfplan \
    | jq -r '.resource_changes[] | [.address, (.change.actions | join(","))] | @tsv'
terraform apply -no-color /tmp/video2-cluster-autoscaler.tfplan \
    >/tmp/video2-cluster-autoscaler-apply.log
grep -E '^(Apply complete|Outputs:[[:alnum:]]+= <sensitive>)' \
    /tmp/video2-cluster-autoscaler-apply.log
yc managed-kubernetes node-group list --format json \
    | jq -r '.[0] | [.name, .status] | @tsv'
kubectl --context "$MANAGED_CONTEXT" get nodes -o wide
BASELINE_NODES="$(kubectl --context "$MANAGED_CONTEXT" get nodes \
    --no-headers | wc -l | tr -d ' ')"
test "$BASELINE_NODES" -eq 2
```

✓ После команды

Ожидая: IAM-токен обновлён в текущем shell; Terraform меняет существующую node group с fixed scale на auto scale `[2..5]`; перед нагрузкой в managed-кластере ровно две Ready-ноды.

🗣 ГОВОРЮ ПЕРЕД PENDING

«Применяю DEVOPS MAY CRY в managed-кластер. У каждой ноды два vCPU, а request каждой реплики ставлю `[1500m]`. На одну ноду помещается только одна такая реплика. Чтобы rolling update не создал промежуточный surge Pod и не запустил autoscaling раньше нужного кадра, здесь делаю контролируемый lab-reset: временно scale в ноль, меняю request и затем сразу запускаю три реплики. Для production-деплойа так делать не надо; здесь это только способ получить воспроизводимое доказательство scheduler-а.

Две ноды принимают две реплики; третья гарантированно остаётся Pending.

Сначала доказываю причину через событие `[Insufficient cpu]`. Высокая загрузка CPU сама по себе не является сигналом Cluster Autoscaler. Сигнал — unschedulable Pod, который смог бы разместиться на новой ноде подходящей группы.

Затем жду два факта: число Ready-нод стало больше baseline, и тот же Pending Pod перешёл в Running. После доказательства возвращаю request `[100m]` и две реплики».

🖥 ПОКАЗАТЬ

- `[devops_may_cry.yaml]`: две replicas, request `[100m]`, PDB и topology spread.
- Контролируемый lab-reset: scale в ноль → patch `[1500m]` → scale до трёх.
- Events Pending Pod.
- В отдельный момент — Yandex Cloud UI, новая VM в node group.
- Финальный `[get pods -o wide]`: третья реплика сидит на новой ноде.

📑 V2-C3-S12-C02 · Создать 1500m Pending и доказать появление новой ноды

```
LAB_DIR="$REPO_ROOT/03_ЧАСТЬ_3_SCALING/32_3.11_cluster_autoscaler"
cd "$LAB_DIR" || exit
kubectl --context "$MANAGED_CONTEXT" apply -f devops_may_cry.yaml
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
    scale deploy/devops-may-cry --replicas=0
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
    rollout status deploy/devops-may-cry --timeout=180s
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab patch deploy devops-may-cry \
    --type=json \
    -
p='[{"op":"replace","path":"/spec/template/spec/containers/0/resources/requests/c
pu","value":"1500m"}]'
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
    scale deploy/devops-may-cry --replicas=3
PENDING_POD=""
for _attempt in $(seq 1 30); do
    PENDING_POD="$(kubectl --context "$MANAGED_CONTEXT" -n traffic-lab get pod \
        -l app=devops-may-cry --field-selector=status.phase=Pending \
        -o jsonpath='{.items[0].metadata.name}' 2>/dev/null || true)"
    test -n "$PENDING_POD" && break
    sleep 2
done
test -n "$PENDING_POD"
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab describe pod "$PENDING_POD" \
    | sed -n '/Events:/, $p'
for _attempt in $(seq 1 60); do
    PHASE="$(kubectl --context "$MANAGED_CONTEXT" -n traffic-lab get pod
"$PENDING_POD" \
        -o jsonpath='{.status.phase}')"
    CURRENT_NODES="$(kubectl --context "$MANAGED_CONTEXT" get nodes \
        --no-headers | wc -l | tr -d ' ')"
    printf 'nodes=%s pod=%s phase=%s\n' "$CURRENT_NODES" "$PENDING_POD" "$PHASE"
```

```
test "$CURRENT_NODES" -gt "$BASELINE_NODES" && test "$PHASE" = "Running" &&
break
sleep 10
done
test "$CURRENT_NODES" -gt "$BASELINE_NODES"
test "$PHASE" = "Running"
kubectl --context "$MANAGED_CONTEXT" get nodes -o wide
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab get pods \
-l app=devops-may-cry -o wide
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab patch deploy devops-may-cry \
--type=json \
-
p='[{"op":"replace","path":"/spec/template/spec/containers/0/resources/requests/c
pu","value":"100m"}]'
```

```
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
scale deploy/devops-may-cry --replicas=2
kubectl --context "$MANAGED_CONTEXT" -n traffic-lab \
rollout status deploy/devops-may-cry --timeout=180s
```

✓ После команды

Ожидая: после контролируемого lab-reset в ноль сразу создаются три реплики с request `1500m`; третья сначала Pending с событием `Insufficient cpu`. Затем число Ready-нод становится больше двух и тот же Pod переходит в Running. В конце request и replicas возвращены к baseline.

🧠 ФИЗИЧЕСКИЙ СМЫСЛ

Scheduler создаёт доказательство нехватки места — Pending с причиной. Cluster Autoscaler моделирует, поможет ли новая нода, просит provider увеличить node group, а после регистрации kubelet scheduler назначает тот же Pod.

⚠️ PRODUCTION NUANCE

Нода может не появиться из-за quota, недоступной зоны, taint, affinity, слишком большого request или неподходящей node group. Поэтому acceptance criterion включает Pending event, изменение node group, Ready-ноду и Running Pod. Одной строки «autoscaling enabled» в Terraform мало.

ПЕРЕХОД

«Теперь понятно, как Kubernetes измеряет, ограничивает и масштабирует приложение. В четвёртой части пройдем путь запроса: DNS, Service, EndpointSlice, kube-proxy, NodePort и внешний HA-вход».

Время: 139:00–143:00

ЭКРАН: итоговая доска Ч3.

ГОВОРЮ

«Соберу цепочку без новых терминов.

Metrics Server дал текущие resource samples. Requests зарезервировали место у scheduler. Limits превратились в cgroup. Memory limit дал OOMKilled, CPU limit — throttling без restart. Kubelet отдельно показал Evicted. HPA увеличил число Pod'ов, VPA в режиме Off посчитал request, Cluster Autoscaler добавил ноду для доказанного Pending.

Это три разных контура. HPA не создаёт VM. VPA не лечит нехватку capacity. Cluster Autoscaler не исправляет неверный request. Когда каждый отвечает на свой сигнал, масштабирование можно проверять по фактам, а не по одному зелёному статусу».

ЧЕКПОЙНТ ПЕРЕД Ч4

- Self-managed context остался `kubespray`.
- Managed context остался `yc-managed`.
- HPA, VPA и временные fault-workload удалены или возвращены к baseline.
- Managed DEVOPS MAY CRY: две реплики, request `100m`.
- Node group остаётся auto scale `2..5`; её cleanup будет в финальном cleanup выпуска.

с4.0 Какой путь сейчас доказываем

Время: 00:00–04:00

ЭКРАН: доска, полный путь запроса; затем браузер DEVOPS MAY CRY.

ГОВОРЮ

«В первом видео путь от браузера до Pod'a мы не разбирали. Сейчас пройдем его по слоям и на каждом найдем объект, который можно проверить.

Сначала разберем TCP, DNS и маршруты. Внутри кластера посмотрим Pod resolv.conf и CoreDNS. Затем свяжем NodePort, Service, EndpointSlice и текущий data plane kube-proxy. В финале добавим внешний этаж: два HAProxy, keepalived, VRRP и один VIP. Остановим активный балансировщик и измерим, что увидел клиент.

Конкретный лабораторный путь такой: браузер → локальный SSH-туннель лабы → private VIP `10.77.0.10:8080` → HAProxy L4 → NodePort `30080` → Service/EndpointSlice → Pod. В production туннеля нет: его место занимает реальный public IP и маршрутизация. Ingress в эту цепочку не вставлен: сначала доказываем базовый L4-маршрут без второго контроллера».

ПЕРЕХОД

«До Kubernetes начну с соединения на одном компьютере. Там три пакета handshake видны без облака и CNI».

Время: 04:00–11:00

ЭКРАН: два окна терминала или один блок с tcpdump в фоне.

ГОВОРЮ

«TCP сначала создаёт соединение. Клиент отправляет SYN, сервер отвечает SYN/ACK, клиент подтверждает ACK. Только после этого идёт HTTP.

Запускаю обычный Python HTTP server на loopback и ограниченный tcpdump. Это не наше production-приложение — здесь нужен только прозрачный TCP-поток».

V2-C4-S01-C01 · Увидеть TCP handshake на loopback

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"
sudo -v
sudo tcpdump -i lo0 -n 'tcp port 8080' &
TCPDUMP_PID=$!
python3 -m http.server 8080 >/tmp/video2-http.log 2>&1 &
HTTP_PID=$!
sleep 1
curl -fsS http://127.0.0.1:8080/ >/dev/null
sleep 1
sudo kill -INT "$TCPDUMP_PID" 2>/dev/null || true
wait "$TCPDUMP_PID" || true
netstat -an | grep -E 'LISTEN|ESTABLISHED|TIME_WAIT' | head
kill "$HTTP_PID" 2>/dev/null || true
```

После команды

Ожидая: в tcpdump видны SYN → SYN/ACK → ACK, затем HTTP-пакеты и закрытие соединения; `netstat` показывает состояния TCP.

ЧТО ЭТО ЗНАЧИТ

`LISTEN` относится к серверному socket, `ESTABLISHED` — к активному соединению, `TIME_WAIT` защищает от старых сегментов после закрытия. Kubernetes эти состояния не отменяет: NodePort и Service всё равно перевозят TCP.

ПЕРЕХОД

«TCP доставляет байты. Без TLS эти байты можно прочитать».

с4.2 Base64 в HTTP не защищает пароль

Время: 11:00–17:00

ЭКРАН: ASCII tcpdump → Authorization header → base64 decode.

ГОВОРЮ

«Basic Authentication кладёт `user:pass` в base64. Это удобная кодировка для текста, не шифрование. Если HTTP идёт без TLS, заголовков виден в capture».

V2-C4-S02-C01 · Доказать, что Basic base64 читается из трафика

```
python3 -m http.server 8080 >/tmp/video2-http.log 2>&1 &
HTTP_PID=$!
sleep 1
sudo -v
sudo tcpdump -i lo0 -n -A 'tcp port 8080' &
TCPDUMP_PID=$!
sleep 1
curl -s http://127.0.0.1:8080/secret -H 'Authorization: Basic dXNlcjpwYXNz'
>/dev/null
sleep 1
sudo kill -INT "$TCPDUMP_PID" 2>/dev/null || true
wait "$TCPDUMP_PID" || true
printf 'dXNlcjpwYXNz' | base64 -d; echo
kill "$HTTP_PID" 2>/dev/null || true
```

После команды

Ожидая: ASCII capture содержит HTTP-заголовок Authorization; base64 декодируется в `user:pass`. Это кодирование, не шифрование.

В ПРОДЕ

TLS защищает канал, но не делает слабый пароль сильным и не отменяет срок жизни credentials. Внутри кластера шифрование трафика зависит от CNI/service mesh и политики, а не появляется само по себе.

ПЕРЕХОД

«Чтобы подключиться по имени, клиент сначала должен получить адрес. Разделю системный resolver и прямой DNS-запрос».

с4.3 hosts, системный resolver и dig — не одно и то же

Время: 17:00–25:00

ЭКРАН: hosts → scutil DNS → dig → временная строка myapp.local.

ГОВОРЮ

«Приложение обычно вызывает системный resolver. Он может учитывать `/etc/hosts`, кэш и настройки ОС. `dig` отправляет DNS-запрос напрямую и `/etc/hosts` не читает.

На минуту добавляю `myapp.local` в hosts. `ping` его находит, `dig` — нет. После доказательства строку сразу удаляю».

V2-C4-S03-C01 · Сравнить системный resolver и прямой DNS-запрос

```
cat /etc/hosts
scutil --dns | sed -n '1,90p'
dig example.com A
dig +trace example.com A
echo '127.0.0.1 myapp.local' | sudo tee -a /etc/hosts
ping -c1 myapp.local
dig myapp.local +short
sudo sed -i '' '/myapp.local/d' /etc/hosts
```

После команды

Ожидая: системный resolver находит `myapp.local` через hosts, а `dig` обращается прямо к DNS и ничего о hosts не знает; строка после доказательства удалена.

ЛОВУШКА

Не оставлять тестовую строку в hosts. И не доказывать работу DNS с помощью `ping`: он показывает итог системного resolver, а не обязательно DNS-ответ.

ПЕРЕХОД

«Теперь поймаю сам DNS-запрос. Resolver и интерфейс возьму из текущей системы, а не захардкожу публичный 8.8.8.8».

с4.4 DNS capture без отключения VPN и перестройки routes

Время: 25:00–31:00

ЭКРАН: фактический resolver, route interface, query/answer на port 53.

ГОВОРЮ

«Сначала читаю, какой nameserver реально использует macOS, затем спрашиваю route, через какой интерфейс до него идти. На машине с VPN это может быть `utun`. Ничего не выключаю и не перенастраиваю — только наблюдаю».

V2-C4-S04-C01 · Снять DNS-пакеты через фактический resolver без изменения VPN

```
DNS_SERVER="$(scutil --dns | awk '/nameserver\[0-9]+\[/{{print $3; exit}}')"  
test -n "$DNS_SERVER"  
IFACE="$(route -n get "$DNS_SERVER" | awk '/interface:/{print $2; exit}')"  
printf 'resolver=%s interface=%s\n' "$DNS_SERVER" "$IFACE"  
sudo -v  
sudo tcpdump -i "$IFACE" -n "host $DNS_SERVER and port 53" &  
TCPDUMP_PID=$!  
sleep 1  
dig @"$DNS_SERVER" example.com A  
sleep 1  
sudo kill -INT "$TCPDUMP_PID" 2>/dev/null || true  
wait "$TCPDUMP_PID" || true
```

После команды

Ожидая: запрос и ответ DNS видны на интерфейсе реального route. VPN, DNS и системные routes не меняются.

ЧТО ЭТО ЗНАЧИТ

DNS отвечает за имя → адрес. Таблица маршрутов отвечает, через какой interface/gateway отправить пакет к этому адресу. Это разные этапы.

ПЕРЕХОД

«Адрес получили. Теперь покажу выбор маршрута на ноутбуке и между двумя VM».

C4.5 Longest prefix match: какой route победил

Время: 31:00–39:00

ЭКРАН: адреса стенда → `route get` → SSH на node2 → private route до node3.

ГОВОРЮ

«Public SSH-адреса рабочего стенда уже записаны в локальном recording env. Private IP не угадываю и не копирую из UI — снимаю командой на самой VM.

`route get` показывает выбранный interface и gateway. Linux-команда `ip route get` делает то же для конкретного destination. Правило выбора — самый длинный подходящий префикс».

V2-C4-S05-C01 · Проследить route с ноутбука и private route между VM

```
set -a
source "$REPO_ROOT/recording/.recording.env"
set +a
SSH_KEY="${VIDEO2_SSH_KEY:?fill VIDEO2_SSH_KEY in recording/.recording.env}"
SSH_USER="${VIDEO2_SSH_USER:-root}"
NODE1_PUBLIC_IP="${NODE1_SSH_HOST:?fill NODE1_SSH_HOST}"
NODE2_PUBLIC_IP="${NODE2_SSH_HOST:?fill NODE2_SSH_HOST}"
NODE3_PRIVATE_IP="$(ssh -i "$SSH_KEY" "$SSH_USER@${NODE3_SSH_HOST:?fill
NODE3_SSH_HOST}" \
    "ip -4 route get 1.1.1.1 | awk '{for (i=1;i<=NF;i++) if (\$i=="src") {print
\$(i+1); exit}}'"")"
route -n get default
route -n get "$NODE1_PUBLIC_IP"
traceroute -m 5 "$NODE1_PUBLIC_IP" || true
ssh -i "$SSH_KEY" "$SSH_USER@$NODE2_PUBLIC_IP" \
    "ip -4 route; ip route get '$NODE3_PRIVATE_IP'"
```


✅ После команды

Ожидаю: ноутбук выбирает interface/gateway по таблице маршрутов; node2 достигает private IP node3 по VPC route. Никакой маршрут вручную не добавляется.

🚧 В ПРОДЕ

Если приложение недоступно только из одной среды, сравнивают route, DNS, security group/ACL и source IP. Ручное добавление маршрута на ноутбуке — не первый диагностический шаг и не замена исправлению инфраструктурного кода.

ПЕРЕХОД

«Переношу тот же вопрос внутрь Pod: какой DNS ему выдали и почему короткое имя Service работает».

с4.6 Pod resolv.conf, CoreDNS, search и ndots

Время: 39:00–49:00

ЭКРАН: debug-client YAML → CoreDNS Pod'ы → `/etc/resolv.conf` клиента.

ГОВОРЮ

«Применяю DEVOPS MAY CRY и отдельный debug-client с `dig` и `curl`. Debug-инструменты не добавляю в distroless production image: для диагностики создаю отдельный одноразовый Pod».

📄 **V2-C4-S06-C01 · Запустить приложение и debug-client, затем открыть Pod resolv.conf**

```
set -a
source "$REPO_ROOT/recording/.recording.env"
set +a
CTX="${SELF_CONTEXT:-kubespray}"
LAB="$REPO_ROOT/04_ЧАСТЬ_4_TRAFFIC/38_4.6_pod_resolv_coredns"
kubectl --context "$CTX" apply -f "$LAB/devops_may_cry.yaml"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
kubectl --context "$CTX" apply -f "$LAB/debug-client.yaml"
kubectl --context "$CTX" -n traffic-lab wait --for=condition=Ready pod/client --
timeout=180s
kubectl --context "$CTX" -n kube-system get pods -l k8s-app=kube-dns -o wide
kubectl --context "$CTX" -n traffic-lab exec client -- cat /etc/resolv.conf
```

✅ После команды

Ожидая: CoreDNS Pod'ы Running; в client видны cluster DNS Service IP, search suffixes и `options ndots:5`.

ПОКАЗАТЬ

- `nameserver` — ClusterIP kube-dns.
- search suffix текущего namespace, `svc` и cluster domain.
- `ndots:5`: имя с меньшим числом точек сначала пробуется через search list.

ГОВОРЮ

«Полное имя Service — `devops-may-cry.traffic-lab.svc.cluster.local`. Короткое `devops-may-cry` работает внутри того же namespace благодаря search suffix».

V2-C4-S06-C02 · Разрешить полное и короткое имя Service через CoreDNS

```
kubectl --context "$CTX" -n traffic-lab exec client -- \
  dig +short devops-may-cry.traffic-lab.svc.cluster.local
kubectl --context "$CTX" -n traffic-lab exec client -- \
  dig +search +short devops-may-cry
```

После команды

Ожидая: оба запроса возвращают один ClusterIP; короткое имя раскрывается через search текущего namespace.

ПЕРЕХОД

«DNS вернул ClusterIP. Но внешний клиент до ClusterIP не маршрутизируется. Для простого входа открою NodePort».

с4.7 NodePort: порт на нодах, а не новый Pod

Время: 49:00–58:00

ЭКРАН: `kubernetes/devops-may-cry/services/nodeport.yaml` → Service → curl с lb1 на private IP node2:30080.

ГОВОРЮ

«NodePort резервирует один порт на Kubernetes-нодах и связывает его с Service. В нашем каноне порт закреплён как `30080`, потому что позже HAProxy будет использовать его как backend.

Public/floating IP здесь нужен только для SSH. С lb1 проверяю NodePort по private IP node2. Ровно так позже пойдёт HAProxy».

V2-C4-S07-C01 · Открыть NodePort и проверить вход через Kubernetes-ноду

```
kubectl --context "$CTX" apply -f "$REPO_ROOT/kubernetes/devops-may-
cry/services/nodeport.yaml"
kubectl --context "$CTX" -n traffic-lab get service devops-may-cry-ha-nodeport -o
wide
NODE_PORT="$(kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry-ha-
nodeport -o jsonpath='{.spec.ports[0].nodePort}')"
NODE2_PRIVATE_IP="$(ssh -i "$SSH_KEY" "$SSH_USER@${NODE2_SSH_HOST:?fill
NODE2_SSH_HOST}" \
    "ip -4 route get 1.1.1.1 | awk '{for (i=1;i<=NF;i++) if (\$i=="src") {print
\$(i+1); exit}}'"")"
ssh -i "$SSH_KEY" "$SSH_USER@${LB1_SSH_HOST:?fill LB1_SSH_HOST}" \
    "curl -fsS http://$NODE2_PRIVATE_IP:$NODE_PORT/readyz"
```

✅ После команды

Ожидая: Service показывает NodePort `30080`; запрос с lb1 на private IP node2 получает readiness JSON. Public/floating IP нужен только SSH-клиенту.

⚠ ЛОВУШКА

Доступность public NodePort зависит от cloud security group. `type: NodePort` сам по себе не открывает firewall провайдера.

ПЕРЕХОД

«NodePort ведёт к Service. Теперь разделю стабильный VIP Service и изменяемый список backend'ов».

с4.8 Service остаётся, Pod IP меняются

Время: 58:00–70:00

ЭКРАН: Service → Pod IP → EndpointSlice → удаление одного Pod.

ГОВОРЮ

«Service хранит ClusterIP, ports и selector. Фактические ready backend-адреса публикуются в EndpointSlice. Поэтому ClusterIP — не IP приложения и не адрес одного Pod».

V2-C4-S08-C01 · Связать стабильный ClusterIP с изменяемыми EndpointSlice

```
kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry -o wide
kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o wide
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry -o wide
kubectl --context "$CTX" -n traffic-lab exec client -- curl -fsS http://devops-
may-cry/readyz
```

После команды

Ожидая: ClusterIP отличается от Pod IP; EndpointSlice содержит ready Pod IP; запрос через имя Service получает HTTP 200.

ГОВОРЮ

«Удаляю один Pod. Deployment создаёт замену с новым IP, EndpointSlice обновляется, а ClusterIP не меняется. Именно эта стабильность позволяет клиентам не знать жизненный цикл реплик».

📑 V2-C4-S08-C02 · Удалить Pod и увидеть замену endpoint без смены ClusterIP

```
SERVICE_IP="$(kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry -o
jsonpath='{.spec.clusterIP}')"
POD="$(kubectl --context "$CTX" -n traffic-lab get pod -l app=devops-may-cry -o
jsonpath='{.items[0].metadata.name}')"
OLD_POD_IP="$(kubectl --context "$CTX" -n traffic-lab get pod "$POD" -o
jsonpath='{.status.podIP}')"
DESIRED="$(kubectl --context "$CTX" -n traffic-lab get deploy devops-may-cry -o
jsonpath='{.spec.replicas}')"
kubectl --context "$CTX" -n traffic-lab delete pod "$POD" --wait=true

ENDPOINT_IPS=""
for _ in $(seq 1 180); do
    READY="$(kubectl --context "$CTX" -n traffic-lab get deploy devops-may-cry \
        -o jsonpath='{.status.readyReplicas}')"
    ENDPOINT_IPS="$(kubectl --context "$CTX" -n traffic-lab get endpointslice \
        -l kubernetes.io/service-name=devops-may-cry \
        -o jsonpath='{range .items[*].endpoints[*]}.{.addresses[0]}{"\n"}{end}')"
    ENDPOINT_COUNT="$(printf '%s\n' "$ENDPOINT_IPS" | awk 'NF {n++} END {print
n+0}')"
    if [ "${READY:-0}" = "$DESIRED" ] \
        && [ "$ENDPOINT_COUNT" -ge "$DESIRED" ] \
        && ! printf '%s\n' "$ENDPOINT_IPS" | grep -Fxq "$OLD_POD_IP"; then
        break
    fi
    sleep 1
done
test "${READY:-0}" = "$DESIRED"
test "$ENDPOINT_COUNT" -ge "$DESIRED"
! printf '%s\n' "$ENDPOINT_IPS" | grep -Fxq "$OLD_POD_IP"
kubectl --context "$CTX" -n traffic-lab get endpointslice \
    -l kubernetes.io/service-name=devops-may-cry -o wide
test "$SERVICE_IP" = "$(kubectl --context "$CTX" -n traffic-lab get svc devops-
```

```
may-cry -o jsonpath='{.spec.clusterIP}')"
kubectl --context "$CTX" -n traffic-lab exec client -- curl -fsS http://devops-
may-cry/readyz
```

✅ После команды

Ожидая: bounded-цикл ждёт полного числа ready endpoints, старый Pod IP исчезает, а HTTP снова даёт 200; ClusterIP остаётся тем же.

ПЕРЕХОД

«Осталось понять, где ClusterIP превращается в выбор endpoint. Не буду заранее говорить iptables или IPVS — сначала читаю текущую конфигурацию kube-proxy».

с4.9 kube-проху: сначала факт, потом объяснение

Время: 70:00–80:00

ЭКРАН: kube-proxy ConfigMap → worker IPVS rules → HTTP proof.

ГОВОРЮ

«У нашего Kubespray-стенда выбран IPVS и включён strictARP. Проверяю это в ConfigMap, затем смотрю правила на workers.

В IPVS-режиме ClusterIP появляется на `kube-ipvs0`, а virtual server содержит real servers из EndpointSlice. В iptables или nftables реализация была бы другой; общий контракт Service и EndpointSlice остаётся».

V2-C4-S09-C01 · Доказать текущий kube-proxy data plane через IPVS

```
SERVICE_IP="$(kubectl --context "$CTX" -n traffic-lab get svc devops-may-cry -o
jsonpath='{.spec.clusterIP}')"
kubectl --context "$CTX" -n kube-system get configmap kube-proxy \
  -o jsonpath='{.data.config\.conf}' | grep -E '^mode:|^strictARP:'
ANSIBLE="$REPO_ROOT/kubespray/.venv/bin/ansible"
INV="$REPO_ROOT/kubespray/inventory/video2/inventory.ini"
"$ANSIBLE" -i "$INV" kube_node --private-key "$SSH_KEY" --become \
  -m ansible.builtin.shell \
  -a "ip -brief address show kube-ipvs0; ipvsadm -Ln | grep -F -A4
'${SERVICE_IP}:80'"
kubectl --context "$CTX" -n traffic-lab exec client -- \
  curl -fsS -o /dev/null -w 'service HTTP %{http_code}\n' http://devops-may-
cry/readyz
```

✅ После команды

Ожидая: config сообщает IPVS; на workers VIP находится на `kube-ipvs0`, virtual server указывает на ready endpoints, HTTP возвращает 200.

🧠 ЧТО ЭТО ЗНАЧИТ

Цепочка внутри кластера теперь доказана четырьмя независимыми фактами: Service VIP, ready endpoints, IPVS virtual server и HTTP 200 от приложения.

ПЕРЕХОД

«Раз эта цепочка понятна, сломаю selector одной опечаткой и найду точку разрыва без перезапуска приложения».

с4.10 DNS работает, Service есть, а трафика всё равно нет

Время: 80:00–91:00

ЭКРАН: broken Service с `nigth-shift` → DNS → пустой EndpointSlice → labels.

ГОВОРЮ

«Создаю Service с опечаткой в selector. DNS-запись и ClusterIP появятся, потому что объект Service валиден. Но ни один Pod label ему не соответствует, поэтому ready endpoints нет».

V2-C4-S10-C01 · Получить Service с DNS и ClusterIP, но без endpoints

```
cat <<'EOF' | kubectl --context "$CTX" apply -f -
apiVersion: v1
kind: Service
metadata:
  name: devops-may-cry-broken
  namespace: traffic-lab
spec:
  selector:
    app: nigth-shift
  ports:
    - port: 80
      targetPort: 8080
EOF
kubectl --context "$CTX" -n traffic-lab exec client -- dig +search +short devops-
may-cry-broken
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-broken -o wide
kubectl --context "$CTX" -n traffic-lab exec client -- \
  curl -m3 -s -o /dev/null -w '%{http_code}\n' devops-may-cry-broken || true
```

✅ После команды

Ожидаю: DNS и ClusterIP существуют, EndpointSlice не содержит ready адресов, HTTP не проходит. Go-приложение при этом остаётся Ready.

🗣️ ГОВОРЮ

«Иду по цепочке с ближайшего проверяемого слоя: DNS → Service selector → Pod labels → EndpointSlice. Ошибка видна до application logs. Исправляю selector, ограниченно жду ready endpoint и повторяю тот же HTTP proof. Контроллеры сходятся не мгновенно, поэтому один ранний `get` не считаю результатом».

📖 V2-C4-S10-C02 · Найти опечатку selector, восстановить endpoints и HTTP

```
kubectl --context "$CTX" -n traffic-lab describe svc devops-may-cry-broken | sed
-n '/Selector:/p'
kubectl --context "$CTX" -n traffic-lab get pods --show-labels
kubectl --context "$CTX" -n traffic-lab patch svc devops-may-cry-broken \
  -p '{"spec":{"selector":{"app":"devops-may-cry"}}}'
READY_ENDPOINTS=0
for _attempt in $(seq 1 30); do
  READY_ENDPOINTS="$(kubectl --context "$CTX" -n traffic-lab get endpointslice \
    -l kubernetes.io/service-name=devops-may-cry-broken \
    -o jsonpath='{range .items[*].endpoints[*]}.{.addresses[0]}{" "}'
    {.conditions.ready}{"\n"}{end}' \
    | grep -c ' true$' || true)"
  test "$READY_ENDPOINTS" -gt 0 && break
  sleep 2
done
test "$READY_ENDPOINTS" -gt 0
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-broken -o wide
HTTP_CODE=000
for _attempt in $(seq 1 30); do
  HTTP_CODE="$(kubectl --context "$CTX" -n traffic-lab exec client -- \
    curl -sS -o /dev/null -w '%{http_code}' devops-may-cry-broken/readyz ||
true)"
  printf 'ready_endpoints=%s http=%s\n' "$READY_ENDPOINTS" "$HTTP_CODE"
  test "$HTTP_CODE" = "200" && break
  sleep 2
done
test "$HTTP_CODE" = "200"
kubectl --context "$CTX" -n traffic-lab delete svc devops-may-cry-broken
```

✅ После команды

Ожидаю: selector `nighth-shift` не совпадает с label; после исправления endpoints появляются и HTTP становится 200; сломанный Service удалён.

🚧 В ПРОДЕ

Успешный `kubectl apply` доказывает только принятие объекта API. Recovery доказывают Ready workload, ready endpoints и запрос клиента. Если контейнер не стартовал из-за image pull или admission dependency, пустые application logs не опровергают инфраструктурную причину.

ПЕРЕХОД

«Для HTTP-хостов, TLS и маршрутов поверх Service обычно добавляют Ingress Controller или Gateway API. В нашей live-схеме сейчас другой слой: внешний L4-балансировщик напрямую ведёт на NodePort».

Время: 91:00–96:00

ЭКРАН: доска, Ingress/Gateway API между внешним LB и Service; команд нет.

ГОВОРЮ

«Ingress — старый удобный API для HTTP/HTTPS-маршрутов. Работает не сам объект, а установленный Ingress Controller. Gateway API разделяет инфраструктурную точку входа, listener и маршруты приложений; это удобнее для команд с разным владением.

Сегодня я не подменяю этой схемой текущую лабораторию. Здесь HAProxy работает на L4 и ведёт сразу в NodePort. Отдельный Ingress/Gateway слой можно вставить между ними, когда нужны host/path routing и TLS termination».

ПЕРЕХОД

«MetalLB уже показал IP для Service внутри Kubernetes. Теперь убегу его и верну в цепочку внешний HA-этаж на lb1/lb2, который мы установили и проверили в части 1. Здесь он нужен повторно как вход полного browser-to-Pod доказательства».

Время: 96:00–118:00 плюс Ansible

ЭКРАН: NodePort → `inventory.example.ini` и graph без hostvars → site.yml → три роли → PLAY RECAP.

ГОВОРЮ

«MetalLB был отдельной практикой L2 внутри Kubernetes. Перед внешней схемой удаляю его Service и chart, чтобы на доске и в кластере не осталось двух разных точек входа с одним названием.

DEVOPS MAY CRY остаётся. Для HA создаю отдельный NodePort `30080` и проверяю ready endpoints».

📄 V2-C4-S11-C01 · Убрать MetalLB и подготовить NodePort для внешней HA-пары

```
set -a
source "$REPO_ROOT/recording/.recording.env"
set +a
CTX="${SELF_CONTEXT:-kubespray}"
HA_DIR="$REPO_ROOT/ansible/external-ha"
INV="$HA_DIR/inventory.ini"
SSH_KEY="${VIDEO2_SSH_KEY:?fill VIDEO2_SSH_KEY in recording/.recording.env}"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
kubectl --context "$CTX" -n traffic-lab delete service devops-may-cry-metallb --
ignore-not-found
helm --kube-context "$CTX" uninstall metallb -n metallb-system || true
kubectl --context "$CTX" apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context "$CTX" -n traffic-lab rollout status deploy/devops-may-cry --
timeout=180s
kubectl --context "$CTX" apply -f "$REPO_ROOT/kubernetes/devops-may-
cry/services/nodeport.yaml"
kubectl --context "$CTX" -n traffic-lab get service devops-may-cry-ha-nodeport -o
wide
kubectl --context "$CTX" -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-ha-nodeport -o wide
```

✅ После команды

Ожидаю: MetalLB Service/chart удалены, приложение Ready, канонический backend — NodePort 30080 с ready endpoints.

🗣 ГОВОРЮ

«Безопасный inventory example и graph показывают пять ролей без реальных hostvars. В рабочем inventory `ansible_host` — public/floating адрес только для SSH, но этот файл в кадре не печатаю. Ansible facts сам снимает private адрес интерфейса; именно

private IP используется для VXLAN между lb1/lb2 и для backend NodePort на node1-node3».

📄 V2-C4-S11-C02 · Проверить inventory, private IP и backend до применения HA

```
sed -n '1,180p' "$HA_DIR/inventory.example.ini"
ansible-inventory -i "$INV" --graph
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.ping
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.setup \
  -a 'filter=ansible_default_ipv4'
ansible-playbook -i "$INV" "$HA_DIR/preflight-backends.yml" --private-key
"$SSH_KEY"
```

✅ После команды

Ожидая: безопасный example и graph показывают lb1/lb2 и node1-node3 без hostvars; реальные floating-адреса не печатаются. Hosts доступны, facts показывают private IP для VXLAN/backends; preflight с LB достигает private NodePort всех Kubernetes-нод.

🖨️ ПОКАЗАТЬ

- `preflight-backends.yml`: отдельная проверка private NodePort с lb1 до применения ролей.
- `site.yml`: facts всех пяти VM → VXLAN → HAProxy → keepalived → итоговый proof.
- `roles/vxlan_l2`: отдельная лабораторная L2-сеть пары LB.
- `haproxy.cfg.j2`: `mode tcp`, три private backend `:30080`, HTTP health check `/readyz`.
- `keepalived.conf.j2`: `vrrp_script`, `interval/fall`, `nopreempt`, VIP `10.77.0.10/24` на `vxlan100`.

🗣️ ГОВОРЮ

«Сначала syntax check. Затем Ansible напрямую повторно применяет те же роли. На сохранённом стенде это проверка идемпотентности: большинство задач должны дать `ok` или `changed=0`. Если стенд был явно сброшен, те же роли восстановят конфигурацию. Ничего не прячется в install-скрипт».

V2-C4-S11-C03 · Прочитать роли и идемпотентно применить HAProxy + keepalived

```
sed -n '1,240p' "$HA_DIR/site.yml"
sed -n '1,220p' "$HA_DIR/roles/vxlan_l2/tasks/main.yml"
sed -n '1,220p' "$HA_DIR/roles/haproxy/templates/haproxy.cfg.j2"
sed -n '1,220p' "$HA_DIR/roles/keepalived/templates/keepalived.conf.j2"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --syntax-check --private-key
"$SSH_KEY"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --private-key "$SSH_KEY"
```

После команды

Ожидая: `PLAY RECAP` без failed. На непрерывном стенде после части 1 большинство задач дают `ok`/`changed=0`; после явного reset те же роли установят конфигурацию заново. HAProxy работает в `mode tcp`, health check делает HTTP `GET /readyz`, keepalived управляет VIP 10.77.0.10 на vxlan100.

ГОВОРЮ

«После зелёного PLAY RECAP проверяю не только systemd. VIP должен быть ровно на одном LB, а HTTP через VIP — 200. Чтобы открыть этот private VIP в браузере, делаю локальный SSH-туннель через lb1. Это только мост учебной сети, а не замена public ingress».

📄 V2-C4-S11-C04 · Доказать сервисы, единственного владельца VIP и HTTP 200

```
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
VIP_OWNERS=()
for host in lb1 lb2; do
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
    | grep -q "$HA_VIP/24"; then
    VIP_OWNERS+=("$host")
  fi
done
test "${#VIP_OWNERS[@]}" -eq 1
printf 'VIP owner: %s\n' "${VIP_OWNERS[0]}"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
  -m ansible.builtin.uri -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
ssh -i "$SSH_KEY" -o ExitOnForwardFailure=yes -N \
  -L "127.0.0.1:18081:$HA_VIP:$HA_PORT" \
  "${VIDEO2_SSH_USER:-root}@${LB1_SSH_HOST:?fill LB1_SSH_HOST}" &
VIP_TUNNEL_PID=$!
printf '%s\n' "$VIP_TUNNEL_PID" > "$HA_DIR/.vip-tunnel.pid"
sleep 2
curl -fsS http://127.0.0.1:18081/readyz
open http://127.0.0.1:18081/
```

✅ После команды

Ожидая: оба сервиса active, VIP есть ровно на одном LB, запрос через VIP возвращает HTTP 200. Браузер открывает тот же private VIP через локальный SSH-туннель; это мост лабы, а не public ingress.

⚠ ОГРАНИЧЕНИЕ

VIP `10.77.0.10` живёт внутри лабораторного VXLAN пары LB. Это не публичный cloud IP. В production внешний адрес, маршрутизация и VRRP/BGP зависят от сети площадки; иногда failover делают облачным LB или DNS, но у DNS другое время переключения и кэширование.

ПЕРЕХОД

«Схема работает. В части 1 мы уже ломали пару как отдельную HA-лабораторию. Здесь повторяю отказ не ради второй установки, а чтобы связать его с полной клиентской цепочкой через браузерный туннель. Снова определяю фактического владельца VIP и останавливаю его HAProxy».

Время: 118:00–130:00

ЭКРАН: owner detection → секундная probe → stop HAProxy → VIP/journal/probe.

ГОВОРЮ

«Из-за `nopreempt` активным может быть lb1 или lb2. Поэтому ищу VIP на обоих и сохраняю имя владельца. Параллельно запускаю секундную HTTP-пробу с lb1».

📋 **V2-C4-S12-C01 · Определить текущего владельца VIP и запустить измеряемую пробу**

```
VIP_OWNERS=()
for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
        -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' | grep -q
"$HA_VIP/24"; then
        VIP_OWNERS+=("$host")
    fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ACTIVE_LB="${VIP_OWNERS[0]}"
printf '%s\n' "$ACTIVE_LB" > "$HA_DIR/.failed-active-host"
printf 'VIP owner before failure: %s\n' "$ACTIVE_LB"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.shell \
    -a "rm -f /tmp/vip-probe.log; nohup sh -c 'for i in \$(seq 1 25); do printf
\"%s \" \"\$(date +%T)\"; curl --max-time 2 -sS -o /dev/null -w \"%
{http_code}\\n\" http://$HA_VIP:$HA_PORT/readyz || echo timeout; sleep 1; done'
>/tmp/vip-probe.log 2>&1 </dev/null &"
```

✅ После команды

Ожидаю: владелец определяется фактом, а не предположением `lb1`; на lb1 в фоне началась секундная HTTP-проба.

🗣 ГОВОРЮ

«Останавливаю только HAProxy активного LB. `track_script` замечает отказ, `keepalived` меняет состояние, второй LB забирает VIP. Bounded-цикл ждёт единственного нового владельца и HTTP 200; фиксированную паузу не угадываю. Фактический разрыв читаю из `probe log`».

📑 V2-C4-S12-C02 · Остановить HAProxy активного LB и доказать переезд VIP

```
ansible -i "$INV" "$ACTIVE_LB" --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl stop haproxy'
NEW_OWNER=''
for _ in $(seq 1 30); do
  VIP_OWNERS=()
  for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
      -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
      | grep -q "$HA_VIP/24"; then
      VIP_OWNERS+=("$host")
    fi
  done
  if [ "${#VIP_OWNERS[@]}" -eq 1 ] \
    && [ "${VIP_OWNERS[0]}" != "$ACTIVE_LB" ] \
    && ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
      -m ansible.builtin.uri \
      -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200" \
      >/dev/null 2>&1; then
    NEW_OWNER="${VIP_OWNERS[0]}"
    break
  fi
  sleep 1
done
test -n "$NEW_OWNER"
printf 'VIP owner after failure: %s\n' "$NEW_OWNER"
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.shell \
  -a "journalctl -u keepalived --since '3 minutes ago' --no-pager | tail -24"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'cat /tmp/vip-probe.log'
```



```
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \  
-m ansible.builtin.uri -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

✅ После команды

Ожидаю: VIP находится на втором LB, журнал фиксирует смену состояния, в probe виден фактический краткий разрыв/timeout и последующие 200; итоговый HTTP снова 200.

🧠 ДОКАЗАТЕЛЬСТВА

Одновременно нужны четыре факта: HAProxy действительно остановлен на прежнем владельце; VIP исчез там и появился на втором; keepralived journal показывает переход; клиент после разрыва снова получает 200.

ПЕРЕХОД

«Демонстрация отказа не закончена, пока стенд не восстановлен».

Время: 130:00–140:00

ЭКРАН: start HAProxy → оба active → VIP один → HTTP 200 → финальная доска.



ГОВОРЮ

«Читаю имя отказавшего LB из локального marker, запускаю HAProxy и повторяю три проверки. Из-за `no-preempt` VIP не обязан вернуться на lb1. Правильный критерий: оба HAProxy и keepalived active, VIP ровно на одном LB, HTTP 200».

📑 V2-C4-S13-C01 · Восстановить отказавший LB без требования вернуть VIP на lb1

```
FAILED_LB="$(cat "$HA_DIR/.failed-active-host")"
[[ "$FAILED_LB" == "lb1" || "$FAILED_LB" == "lb2" ]]
ansible -i "$INV" "$FAILED_LB" --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl start haproxy'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
VIP_OWNERS=()
for host in lb1 lb2; do
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
    | grep -q "$HA_VIP/24"; then
    VIP_OWNERS+=("$host")
  fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
  -m ansible.builtin.uri -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
curl -fsS http://127.0.0.1:18081/readyz
VIP_TUNNEL_PID="$(cat "$HA_DIR/.vip-tunnel.pid" 2>/dev/null || true)"
[[ -z "$VIP_TUNNEL_PID" ]] || kill "$VIP_TUNNEL_PID" 2>/dev/null || true
rm -f "$HA_DIR/.vip-tunnel.pid"
rm -f "$HA_DIR/.failed-active-host"
kubectl --context "$CTX" -n traffic-lab delete pod client --ignore-not-found
```

✅ После команды

Ожидая: HAProxy и keepalived active на обоих LB, VIP ровно на одном, HTTP 200 и с LB, и через браузерный туннель. Из-за `no-preempt` VIP не обязан вернуться на lb1; туннель и debug-client удалены.

ГОВОРЮ

«Теперь весь лабораторный маршрут подкреплён фактами. Браузер идёт через локальный SSH-туннель к private VIP. keepalived держит один адрес на живом внешнем LB. HAProxy L4 выбирает healthy private NodePort. Kubernetes Service связывает этот вход с ready адресами EndpointSlice. kube-proxy реализует текущий IPVS data plane. На конце отвечает Pod DEVOPS MAY CRY. В production вместо SSH-туннеля будет public адрес и реальная маршрутизация.

Если запрос пропал, не надо перезапускать всё подряд. Идём по тем же точкам: DNS и route → VIP owner → HAProxy health → NodePort → Service selector → EndpointSlice → Pod readiness. На каждом шаге есть команда и наблюдаемый результат».

ФИНАЛ

«Мы начали с Linux-машин и дошли до отказоустойчивого внешнего входа. Между ними были Terraform, Ansible, Kubespray, Scheduler, cgroups, autoscaling, DNS и Service data plane. Все исходники и лаборатории лежат в одном репозитории; команды можно повторить без скрытых install-скриптов».